Case Study 1: University Scholarship Allocation

- **1. Problem Statement**
- Design a system that evaluates applicants using aggregate marks, subject-wise minimum scores, attendance, and
- family income criteria. The system must rank eligible students and resolve ties using predefined tie-breaking rules.

- **2. Objectives**
- - Evaluate student eligibility based on multi-variable criteria.
- - Apply sorting and custom tie-breaking logic.
- - Output a ranked list of eligible candidates for scholarship allocation.
- **3. Assumptions**
- - Aggregate marks are out of 500.
- - Attendance is recorded as a percentage (0 to 100).
- - Annual income is expressed in INR.
- - Minimum threshold: Aggregate ≥ 400, Subject Min ≥ 70, Attendance $\ge 85\%$, Annual Income $\le 6,00,000$.
- - Tie-breaking priority: 1. Higher Aggregate Marks, 2. Higher Attendance, 3. Lower Annual Income.

4. Algorithm

1. Initialize a list of student records containing (Name, Aggregate, Subject Min, Attendance, Income).
2. Create an empty list `eligible_students`.
3. Loop through each student in the list:
   a. Check if Aggregate ≥ 400, Subject Min ≥ 70, Attendance ≥ 85, and Income ≤ 600000.
   b. If all conditions are met, append student data to `eligible_students`.
4. Sort `eligible_students` using a multi-key comparison: primary = Aggregate (descending), secondary = Attendance (descending), tertiary = Income (ascending).
5. Print ranked eligible students.

5. Pseudocode

```text
START
  SET students = [ ("Alice", 450, 75, 90, 400000), ("Bob", 450,
72, 92, 350000), ("Charlie", 390, 65, 88, 300000) ]
  SET eligible = []

  FOR EACH student IN students DO
    IF student.marks >= 400 AND student.min_score >= 70
AND student.attendance >= 85 AND student.income <=
600000 THEN
      APPEND student TO eligible
    ENDIF
  ENDFOR

  SORT eligible BY (marks DESC, attendance DESC, income ASC)

  PRINT "Ranked Scholarship Recipients:"
  FOR rank, student IN ENUMERATE(eligible) DO
    PRINT rank + 1, student
  ENDFOR
END
```

6. Python Source Code

```python
def evaluate_scholarships(students):
    eligible_students = []

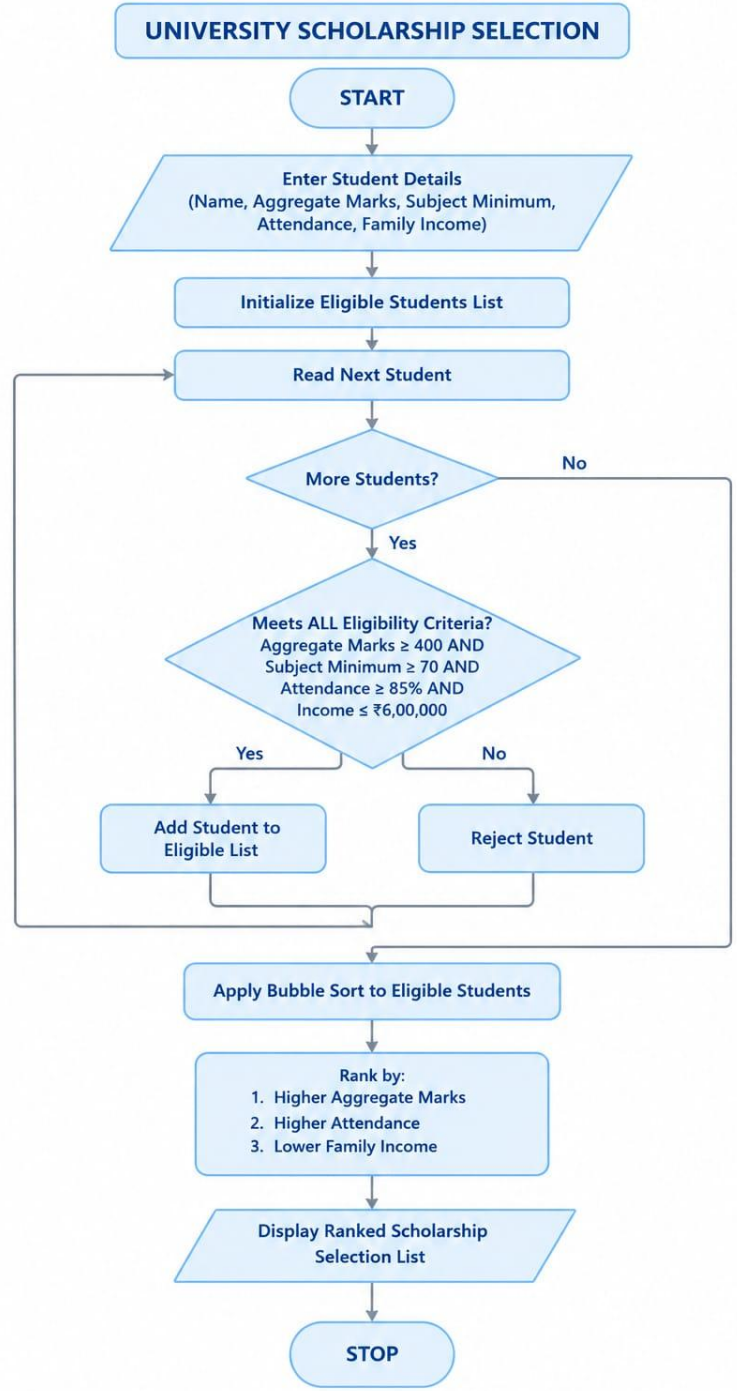
    for student in students:
        name, agg_marks, sub_min, attendance, income = student
        if agg_marks >= 400 and sub_min >= 70 and attendance >= 85
and income <= 600000:
            eligible_students.append(student)


 n = len(eligible_students)
     for i in range(n):
         for j in range(0, n - i - 1):
             s1 = eligible_students[j]
             s2 = eligible_students[j + 1]
```

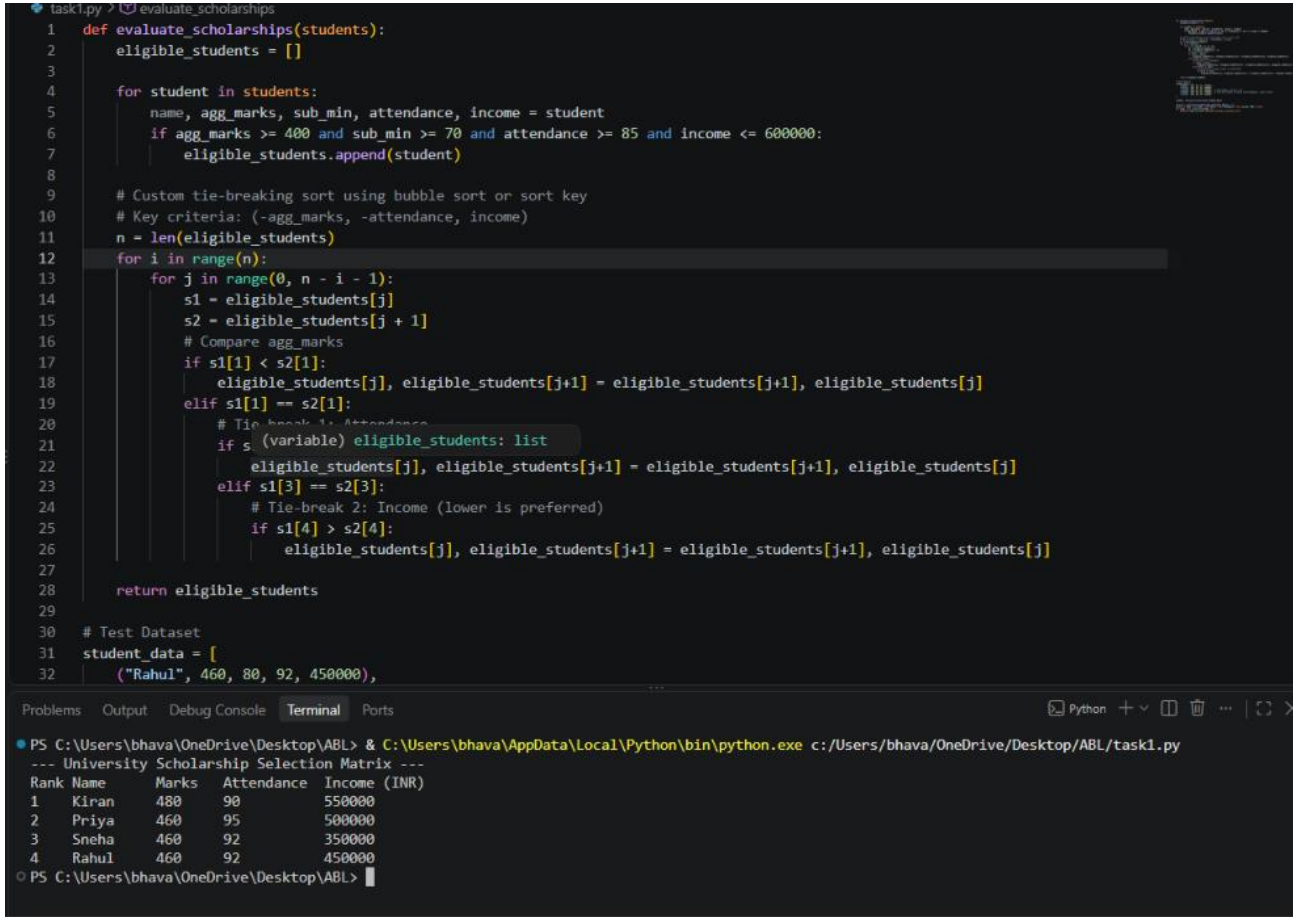
```
if s1[1] < s2[1]:
            eligible_students[j], eligible_students[j+1] =
eligible_students[j+1], eligible_students[j]
         elif s1[1] == s2[1]:
            # Tie-break 1: Attendance
            if s1[3] < s2[3]:
               eligible_students[j], eligible_students[j+1] =
eligible_students[j+1], eligible_students[j]
            elif s1[3] == s2[3]:
               # Tie-break 2: Income (lower is preferred)
               if s1[4] > s2[4]:
                  eligible_students[j], eligible_students[j+1] =
eligible_students[j+1], eligible_students[j]
    return eligible_students
    # Test Dataset
    student_data = [
       ("Rahul", 460, 80, 92, 450000),
       ("Priya", 460, 75, 95, 500000),
       ("Anish", 420, 68, 88, 300000),  # Ineligible: sub_min < 70
       ("Sneha", 460, 82, 92, 350000),  # Tied with Rahul on marks
    and attendance, lower income
       ("Kiran", 480, 88, 90, 550000)
    ]
```

```
ranked = evaluate_scholarships(student_data)


print("--- University Scholarship Selection Matrix ---")
print(f"{'Rank':<5}{'Name':<10}{'Marks':<8}{'Attendance':<12}
{'Income (INR)':<12}")
for rank, s in enumerate(ranked, 1):
    print(f"{rank:<5}{s[0]:<10}{s[1]:<8}{s[3]:<12}{s[4]:<12}")
```


UNIVERSITY SCHOLARSHIP SELECTION
START
Enter Student Details
(Name, Aggregate Marks, Subject Minimum,
Attendance, Family Income)
Initialize Eligible Students List
Read Next Student
More Students?
No
Yes
Meets ALL Eligibility Criteria?
Aggregate Marks ≥ 400 AND
Subject Minimum ≥ 70 AND
Attendance ≥ 85% AND
Income ≤ ₹6,00,000
Yes
No
Add Student to
Eligible List
Reject Student
Apply Bubble Sort to Eligible Students
Rank by:
1. Higher Aggregate Marks
2. Higher Attendance
3. Lower Family Income
Display Ranked Scholarship
Selection List
STOP

Screen shot code and output



```python
def evaluate_scholarships(students):
    eligible_students = []

    for student in students:
        name, agg_marks, sub_min, attendance, income = student
        if agg_marks >= 400 and sub_min >= 70 and attendance >= 85 and income <= 600000:
            eligible_students.append(student)

    # Custom tie-breaking sort using bubble sort or sort key
    # Key criteria: (-agg_marks, -attendance, income)
    n = len(eligible_students)
    for i in range(n):
        for j in range(0, n - i - 1):
            s1 = eligible_students[j]
            s2 = eligible_students[j + 1]
            # Compare agg_marks
            if s1[1] < s2[1]:
                eligible_students[j], eligible_students[j+1] = eligible_students[j+1], eligible_students[j]
            elif s1[1] == s2[1]:
                # Tie-break 1: Attendance
                if s
                    (variable) eligible_students: list
                    eligible_students[j], eligible_students[j+1] = eligible_students[j+1], eligible_students[j]
                elif s1[3] == s2[3]:
                    # Tie-break 2: Income (lower is preferred)
                    if s1[4] > s2[4]:
                        eligible_students[j], eligible_students[j+1] = eligible_students[j+1], eligible_students[j]

    return eligible_students

# Test Dataset
student_data = [
    ("Rahul", 460, 80, 92, 450000),
```

```
Problems  Output  Debug Console  Terminal  Ports                                    Python

PS C:\Users\bhava\OneDrive\Desktop\ABL> & C:\Users\bhava\AppData\Local\Python\bin\python.exe c:/Users/bhava/OneDrive/Desktop/ABL/task1.py
--- University Scholarship Selection Matrix ---
Rank Name      Marks   Attendance  Income (INR)
1    Kiran     480     90          550000
2    Priya     460     95          500000
3    Sneha     460     92          350000
4    Rahul     460     92          450000
PS C:\Users\bhava\OneDrive\Desktop\ABL>
```

8. Conclusion

The University Scholarship Allocation script accurately filters applicants against multi-layered eligibility constraints and resolves equal-score ties using prioritized secondary criteria.

Case Study 2: Progressive Electricity Billing

1. Problem Statement

- Calculate household electricity bills using progressive consumption slabs, fixed charges, surcharges, and rebates. Compare bills across multiple months to identify unusual consumption spikes.

2. Objectives

- - Compute billing amounts using tiered pricing logic.
- - Process multi-month data using lists and conditional structures.
- - Detect abnormal increases exceeding a threshold percentage.
- **3. Assumptions**
- - **Slabs:**
- - 0–100 units: ₹3.0/unit
- - 101–300 units: ₹5.0/unit
- - >300 units: ₹8.0/unit
- - Fixed Charge: ₹150 per month.
- - Surcharge: 5% if total bill exceeds ₹2000.
- - Rebate: 2% if consumption is under 100 units.
- - An increase of $\ge 30\%$ over the previous month is flagged as unusual.

4. Algorithm

1. Define a function `calculate_bill(units)` that calculates costs across slabs and adds fixed charges/penalties.
2. For each month's consumption:
   a. Compute monthly bill using `calculate_bill`.
   b. Compare consumption with the previous month.
   c. If percentage increase $\ge 30\%$, flag as an unusual increase.
3. Output the monthly bill breakdown and alerts.

5. Pseudocode

```
START
  FUNCTION calculate_monthly_bill(units):
    SET bill = 150 (Fixed charge)
    IF units <= 100 THEN
      bill = bill + (units * 3.0) - rebate
    ELSE IF units <= 300 THEN
      bill = bill + (100 * 3.0) + ((units - 100) * 5.0)
    ELSE
      bill = bill + (100 * 3.0) + (200 * 5.0) + ((units - 300) * 8.0)
    ENDIF
    IF bill > 2000 THEN bill = bill + (0.05 * bill) ENDIF
    RETURN bill
  END FUNCTION

  SET monthly_units = [80, 150, 280, 400, 420]
    FOR month 1 TO length(monthly_units):
      Compute Bill
      IF month > 1 AND percent_increase >= 30% THEN
        PRINT alert flag
      ENDIF
    ENDFOR
  END
```

6. Python Source Code

```python
def compute_electricity_bill(units):
    fixed_charge = 150.0
    energy_charge = 0.0

if units <= 100:
        energy_charge = units * 3.0
    elif units <= 300:
        energy_charge = (100 * 3.0) + ((units - 100) * 5.0)
    else:
        energy_charge = (100 * 3.0) + (200 * 5.0) + ((units - 300) *
8.0)
    total = fixed_charge + energy_charge

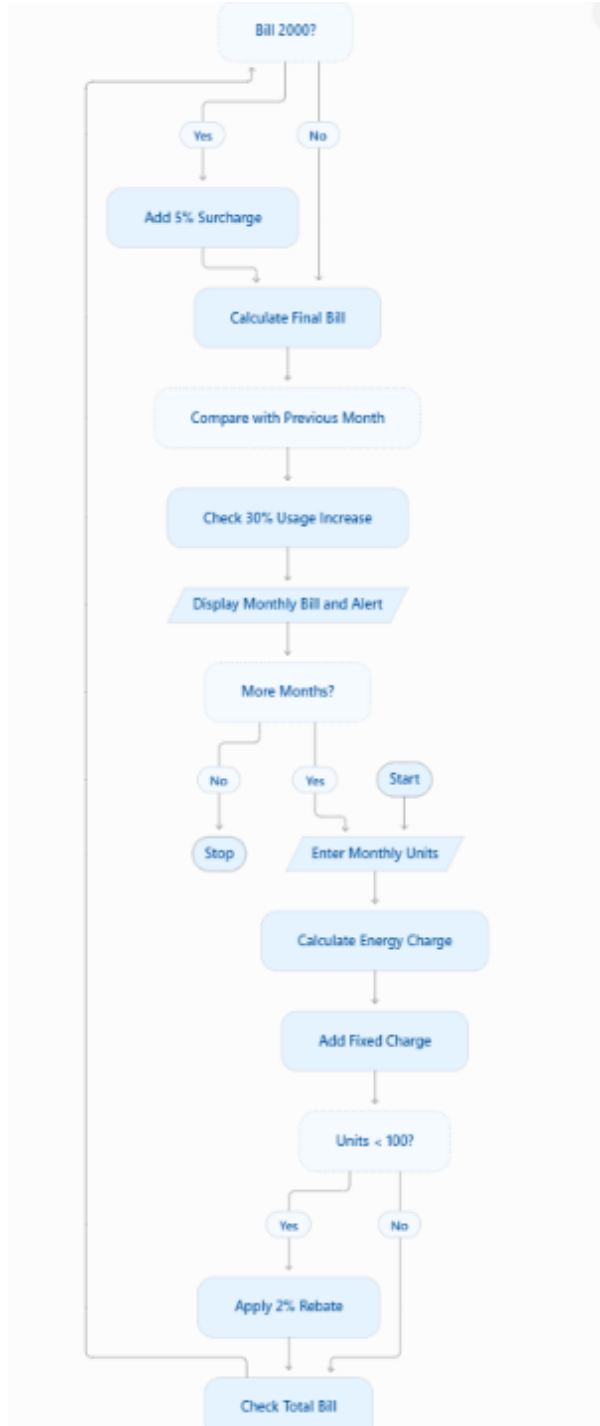
if units < 100:
        total -= total * 0.02

 if total > 2000:
         total += total * 0.05
     return total
 months = ["Jan", "Feb", "Mar", "Apr", "May"]
 monthly_units = [80, 120, 250, 350, 200]
 previous_units = none
```

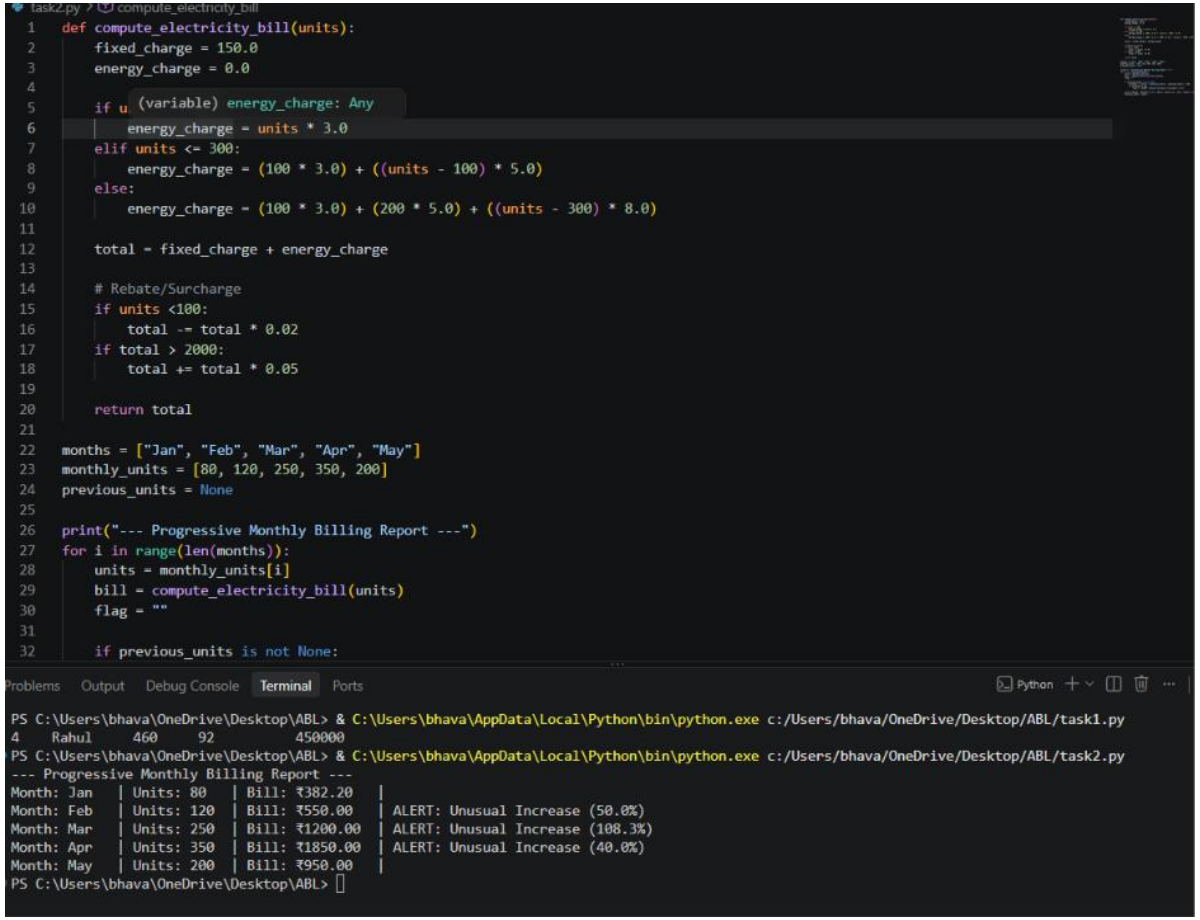
```
print("--- Progressive Monthly Billing Report ---")
for i in range(len(monthly_consumption)):
    units = monthly_consumption[i]
    bill = compute_electricity_bill(units)
    flag = ""


if previous_units is not None:
        increase = ((units - previous_units) / previous_units) *
100
        if increase >= 30:
            flag = f"ALERT: Unusual Increase ({increase:.1f}%)"

    print(f"Month: {months[i]:<5} | Units: {units:<4} | Bill:
₹{bill:<8.2f} | {flag}")
    previous_units = units
```


Bill 2000?
Yes
No
Add 5% Surcharge
Calculate Final Bill
Compare with Previous Month
Check 30% Usage Increase
Display Monthly Bill and Alert
More Months?
No
Yes
Start
Stop
Enter Monthly Units
Calculate Energy Charge
Add Fixed Charge
Units < 100?
Yes
No
Apply 2% Rebate
Check Total Bill

Screen shot code and output



```python
def compute_electricity_bill(units):
    fixed_charge = 150.0
    energy_charge = 0.0

    if u (variable) energy_charge: Any
        energy_charge = units * 3.0
    elif units <= 300:
        energy_charge = (100 * 3.0) + ((units - 100) * 5.0)
    else:
        energy_charge = (100 * 3.0) + (200 * 5.0) + ((units - 300) * 8.0)

    total = fixed_charge + energy_charge

    # Rebate/Surcharge
    if units <100:
        total -= total * 0.02
    if total > 2000:
        total += total * 0.05

    return total

months = ["Jan", "Feb", "Mar", "Apr", "May"]
monthly_units = [80, 120, 250, 350, 200]
previous_units = None

print("--- Progressive Monthly Billing Report ---")
for i in range(len(months)):
    units = monthly_units[i]
    bill = compute_electricity_bill(units)
    flag = ""

    if previous_units is not None:
```

```
Problems  Output  Debug Console  Terminal  Ports                                                    Python

PS C:\Users\bhava\OneDrive\Desktop\ABL> & C:\Users\bhava\AppData\Local\Python\bin\python.exe c:/Users/bhava/OneDrive/Desktop/ABL/task1.py
4    Rahul     460     92          450000
PS C:\Users\bhava\OneDrive\Desktop\ABL> & C:\Users\bhava\AppData\Local\Python\bin\python.exe c:/Users/bhava/OneDrive/Desktop/ABL/task2.py
--- Progressive Monthly Billing Report ---
Month: Jan   | Units: 80   | Bill: ₹382.20   |
Month: Feb   | Units: 120  | Bill: ₹550.00   | ALERT: Unusual Increase (50.0%)
Month: Mar   | Units: 250  | Bill: ₹1200.00  | ALERT: Unusual Increase (108.3%)
Month: Apr   | Units: 350  | Bill: ₹1850.00  | ALERT: Unusual Increase (40.0%)
Month: May   | Units: 200  | Bill: ₹950.00   |
PS C:\Users\bhava\OneDrive\Desktop\ABL>
```

8. Conclusion

The progressive electricity billing program successfully processes multi-rate slab calculations and detects sudden jumps in consumption

Case Study 3: Hospital Billing and Patient Classification

1. Problem Statement

Process consultation, diagnostic, and treatment charges, apply conditional concessions based on patient categories, validate billing constraints, and classify patients.

2. Objectives

- - Compute medical charges with variable fee structures.
- - Apply category-specific discount concessions (e.g., Senior Citizen, BPL, General).
- - Classify patient accounts into payment status groups.

• 3. Assumptions

- - Patient Categories: "Senior Citizen" (20% discount), "BPL" (50% discount), "General" (0% discount).
- - Fixed Charges: Consultation = ₹500.
- - Mandatory tax/service charge = 5% for General category if total $> ₹10,000$.

4. Algorithm

1. Input patient profile containing Consultation, Diagnostics cost, Treatment cost, and Category.
2. Sum basic charges = Consultation + Diagnostics + Treatment.
3. Determine applicable discount percentage based on Category.
4. Calculate net billing amount and apply surcharge if applicable.
5. Classify billing status as "High-Value", "Standard", or "Subsidized".

5. Pseudocode

```text
START
  READ consultation, diagnostics, treatment, category
  SET subtotal = consultation + diagnostics + treatment

  IF category == "BPL" THEN
    SET discount = 0.50 * subtotal
  ELSE IF category == "Senior Citizen" THEN
    SET discount = 0.20 * subtotal
  ELSE
    SET discount = 0
  ENDIF

  SET net_bill = subtotal - discount
    IF category == "General" AND net_bill > 10000 THEN
      net_bill = net_bill * 1.05
    ENDIF

    PRINT final invoice details
  END
```

6. Python Source Code

```python
def process_patient_billing(patient_id, name, diag_cost,
treat_cost, category):
    consultation_fee = 500.0
    subtotal = consultation_fee + diag_cost + treat_cost

    if category == "BPL":
        discount = 0.50 * subtotal
    elif category == "Senior Citizen":
        discount = 0.20 * subtotal
    else:
        discount = 0.0

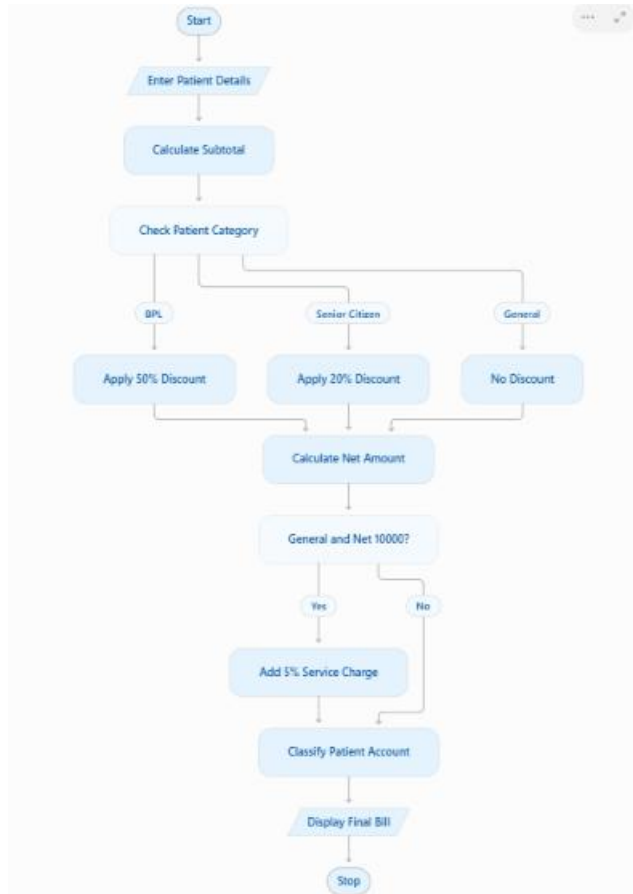
    net = subtotal - discount
if category == "General" and net > 10000:
        net += net * 0.05  # Service charge

    # Classification
    if category == "BPL":
        classification = "Subsidized Care"
    elif net > 15000:
        classification = "High-Value Account"
```

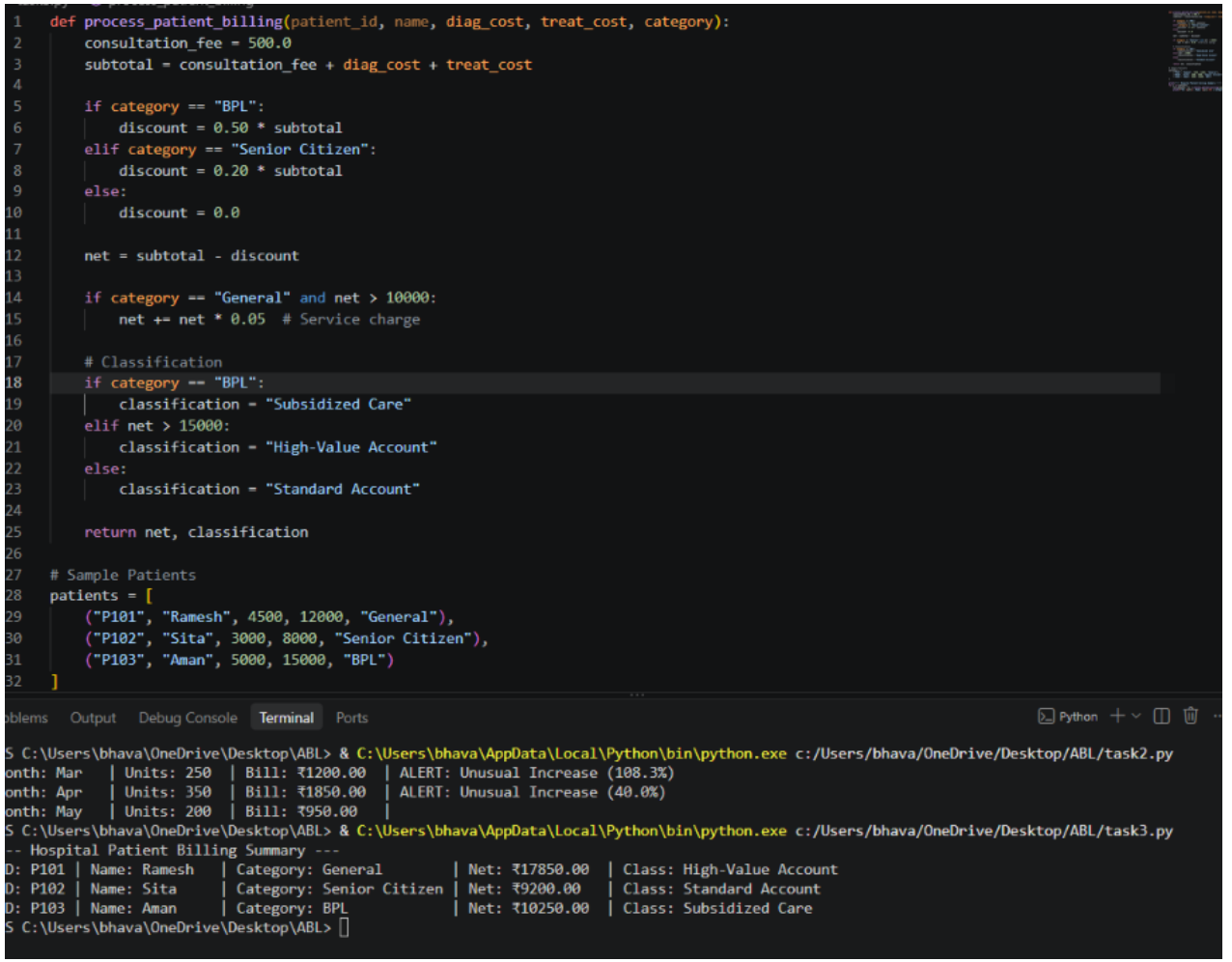
```
        else:
        classification = "Standard Account"

    return net, classification

# Sample Patients
patients = [
    ("P101", "Ramesh", 4500, 12000, "General"),
    ("P102", "Sita", 3000, 8000, "Senior Citizen"),
    ("P103", "Aman", 5000, 15000, "BPL")
]
print("--- Hospital Patient Billing Summary ---")
for p in patients:
    net_payable, cls = process_patient_billing(p[0], p[1], p[2],
p[3], p[4])
    print(f"ID: {p[0]} | Name: {p[1]:<8} | Category: {p[4]:<14} |
Net: ₹{net_payable:<9.2f} | Class: {cls}")
```


Start
Enter Patient Details
Calculate Subtotal
Check Patient Category
BPL
Senior Citizen
General
Apply 50% Discount
Apply 20% Discount
No Discount
Calculate Net Amount
General and Net 10000?
Yes
No
Add 5% Service Charge
Classify Patient Account
Display Final Bill
Stop

Screen shot code and output



```python
def process_patient_billing(patient_id, name, diag_cost, treat_cost, category):
    consultation_fee = 500.0
    subtotal = consultation_fee + diag_cost + treat_cost

    if category == "BPL":
        discount = 0.50 * subtotal
    elif category == "Senior Citizen":
        discount = 0.20 * subtotal
    else:
        discount = 0.0

    net = subtotal - discount

    if category == "General" and net > 10000:
        net += net * 0.05  # Service charge

    # Classification
    if category == "BPL":
        classification = "Subsidized Care"
    elif net > 15000:
        classification = "High-Value Account"
    else:
        classification = "Standard Account"

    return net, classification

# Sample Patients
patients = [
    ("P101", "Ramesh", 4500, 12000, "General"),
    ("P102", "Sita", 3000, 8000, "Senior Citizen"),
    ("P103", "Aman", 5000, 15000, "BPL")
]
```

```
oblems  Output  Debug Console  Terminal  Ports                                                    Python

S C:\Users\bhava\OneDrive\Desktop\ABL> & C:\Users\bhava\AppData\Local\Python\bin\python.exe c:/Users/bhava/OneDrive/Desktop/ABL/task2.py
onth: Mar   | Units: 250  | Bill: ₹1200.00  | ALERT: Unusual Increase (108.3%)
onth: Apr   | Units: 350  | Bill: ₹1850.00  | ALERT: Unusual Increase (40.0%)
onth: May   | Units: 200  | Bill: ₹950.00   |
S C:\Users\bhava\OneDrive\Desktop\ABL> & C:\Users\bhava\AppData\Local\Python\bin\python.exe c:/Users/bhava/OneDrive/Desktop/ABL/task3.py
-- Hospital Patient Billing Summary ---
D: P101 | Name: Ramesh   | Category: General        | Net: ₹17850.00  | Class: High-Value Account
D: P102 | Name: Sita     | Category: Senior Citizen | Net: ₹9200.00   | Class: Standard Account
D: P103 | Name: Aman     | Category: BPL            | Net: ₹10250.00  | Class: Subsidized Care
S C:\Users\bhava\OneDrive\Desktop\ABL>
```

8. Conclusion

The script accurately calculates tiered patient invoices and assigns proper classification according to insurance and socio-economic concession policies.

Case Study 4: Employee Payroll and Deduction Analysis

- **1. Problem Statement**
- Calculate gross salary from basic pay, allowances, overtime, and incentives. Apply progressive deductions (Tax, PF) to produce a net salary statement.
- **2. Objectives**
- - Compute dynamic gross salary components.
- - Calculate progressive tax deductions based on income brackets.
- - Display an itemized salary slip summary.
- **3. Assumptions**
- - HRA = 20% of Basic, DA = 10% of Basic.
- - Overtime rate = ₹200 per hour.
- - PF Deduction = 12% of Basic.
- - Income Tax Brackets (on Gross):
- - Gross $\le ₹50,000$: 0%
- - $₹50,001 - ₹1,00,000$: 10%
- - $> ₹1,00,000$: 20%

4. Algorithm

1. Compute HRA = 0.20 * Basic, DA = 0.10 * Basic.
2. Compute Overtime Pay = Overtime Hours * 200.
3. Compute Gross Salary = Basic + HRA + DA + Overtime Pay + Incentive.
4. Calculate PF = 0.12 * Basic.
5. Calculate Tax based on gross salary income slab.
6. Compute Net Salary = Gross Salary - (PF + Tax).

5. Pseudocode

```text
START
   READ basic, ot_hours, incentive
   hra = 0.20 * basic
   da = 0.10 * basic
   ot_pay = ot_hours * 200
   gross = basic + hra + da + ot_pay + incentive
   pf = 0.12 * basic
   IF gross <= 50000 THEN tax = 0
   ELSE IF gross <= 100000 THEN tax = 0.10 * gross
   ELSE tax = 0.20 * gross

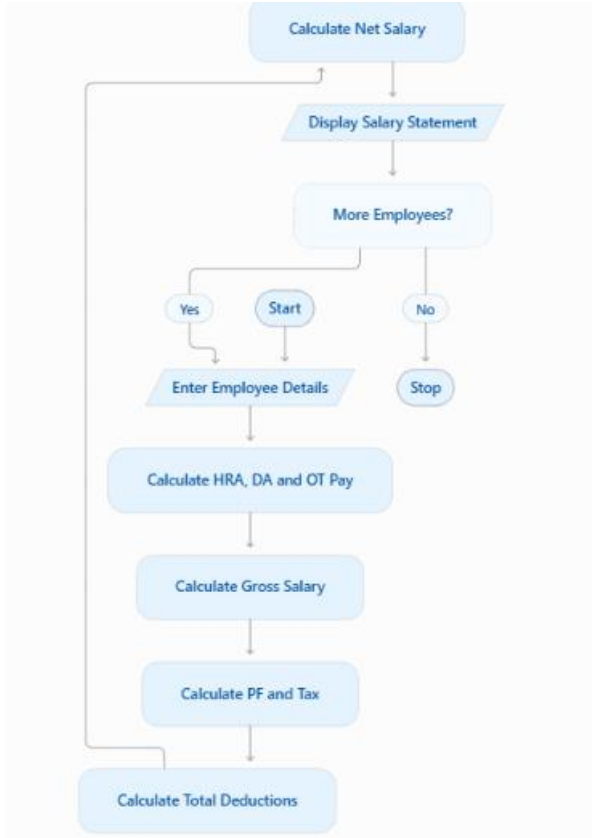
   net_salary = gross - (pf + tax)
   PRINT gross, total_deductions, net_salary
END
```

6. Python Source Code

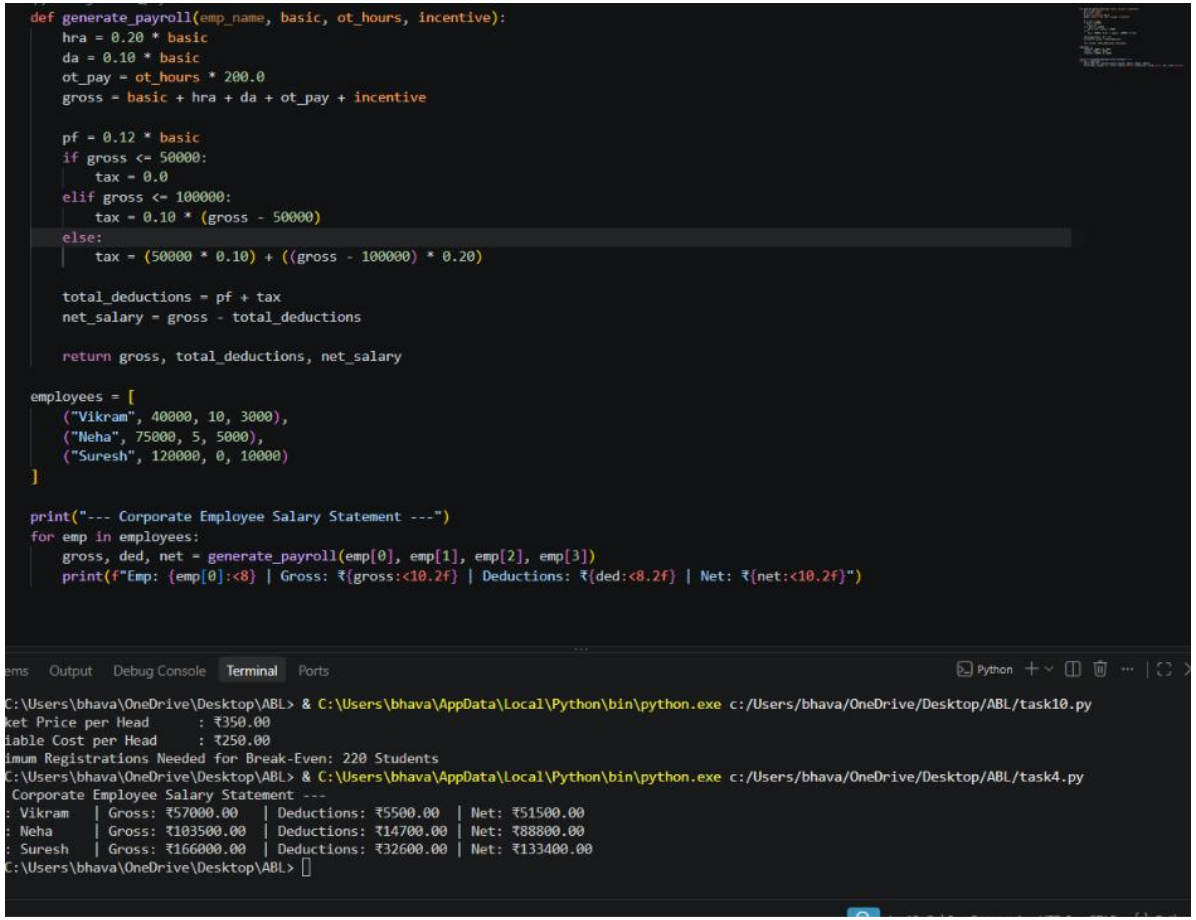
```python
def generate_payroll(emp_name, basic, ot_hours,
incentive):
    hra = 0.20 * basic
    da = 0.10 * basic
    ot_pay = ot_hours * 200.0
    gross = basic + hra + da + ot_pay + incentive
    pf = 0.12 * basic
    if gross <= 50000:
        tax = 0.0
    elif gross <= 100000:
        tax = 0.10 * (gross - 50000)
    else:
        tax = (50000 * 0.10) + ((gross - 100000) * 0.20)

        total_deductions = pf + tax
            net_salary = gross - total_deductions
            return gross, total_deductions, net_salary
        employees = [
            ("Vikram", 40000, 10, 3000),
            ("Neha", 75000, 5, 5000),
            ("Suresh", 120000, 0, 10000)
        ]
```

```
print("--- Corporate Employee Salary Statement ---")
for emp in employees:
    gross, ded, net = generate_payroll(emp[0], emp[1],
emp[2], emp[3])
    print(f"Emp: {emp[0]:<8} | Gross: ₹{gross:<10.2f} |
Deductions: ₹{ded:<8.2f} | Net: ₹{net:<10.2f}")
```


Calculate Net Salary
Display Salary Statement
More Employees?
Yes
Start
No
Enter Employee Details
Stop
Calculate HRA, DA and OT Pay
Calculate Gross Salary
Calculate PF and Tax
Calculate Total Deductions

Screen shot code and output



```
def generate_payroll(emp_name, basic, ot_hours, incentive):
    hra = 0.20 * basic
    da = 0.10 * basic
    ot_pay = ot_hours * 200.0
    gross = basic + hra + da + ot_pay + incentive

    pf = 0.12 * basic
    if gross <= 50000:
        tax = 0.0
    elif gross <= 100000:
        tax = 0.10 * (gross - 50000)
    else:
        tax = (50000 * 0.10) + ((gross - 100000) * 0.20)

    total_deductions = pf + tax
    net_salary = gross - total_deductions

    return gross, total_deductions, net_salary

employees = [
    ("Vikram", 40000, 10, 3000),
    ("Neha", 75000, 5, 5000),
    ("Suresh", 120000, 0, 10000)
]

print("--- Corporate Employee Salary Statement ---")
for emp in employees:
    gross, ded, net = generate_payroll(emp[0], emp[1], emp[2], emp[3])
    print(f"Emp: {emp[0]:<8} | Gross: ₹{gross:<10.2f} | Deductions: ₹{ded:<8.2f} | Net: ₹{net:<10.2f}")
```

```
ems  Output  Debug Console  Terminal  Ports                                                    Python

C:\Users\bhava\OneDrive\Desktop\ABL> & C:\Users\bhava\AppData\Local\Python\bin\python.exe c:/Users/bhava/OneDrive/Desktop/ABL/task10.py
ket Price per Head      : ₹350.00
iable Cost per Head     : ₹250.00
imum Registrations Needed for Break-Even: 220 Students
C:\Users\bhava\OneDrive\Desktop\ABL> & C:\Users\bhava\AppData\Local\Python\bin\python.exe c:/Users/bhava/OneDrive/Desktop/ABL/task4.py
 Corporate Employee Salary Statement ---
: Vikram   | Gross: ₹57000.00   | Deductions: ₹5500.00  | Net: ₹51500.00
: Neha     | Gross: ₹103500.00  | Deductions: ₹14700.00 | Net: ₹88800.00
: Suresh   | Gross: ₹166000.00  | Deductions: ₹32600.00 | Net: ₹133400.00
C:\Users\bhava\OneDrive\Desktop\ABL>
```

8. Conclusion

The employee payroll module evaluates progressive tax brackets and statutory contributions to determine final employee net compensation.

Case Study 5: Multi-Plan Telecommunications Cost Optimization

- **1. Problem Statement**
- Evaluate several mobile plans against expected user data and calling usage. Calculate excess charges and identify the most economical eligible plan.
- **2. Objectives**
- - Compare fixed monthly plan rentals against actual usage patterns.
- - Calculate overage rates for calls and data.
- - Recommend the optimal cost-effective plan.
- **3. Assumptions**
- - ****Plan A:**** Rental ₹299, 1.5 GB/day, Unlimited calls. Excess data: ₹10/GB.
- - ****Plan B:**** Rental ₹399, 2.0 GB/day, Unlimited calls. Excess data: ₹8/GB.
- - ****Plan C:**** Rental ₹199, 1.0 GB/day, 100 mins/day calls. Excess calls: ₹0.50/min, Excess data: ₹15/GB.
- - Duration = 28 days.
-

4. Algorithm

1. For each plan option, calculate base allowance (e.g., total data = daily allowance * 28).
2. Calculate total extra data and call minutes used beyond allowance.
3. Compute total cost = Base Rental + (Extra Data * Rate) + (Extra Calls * Rate).
4. Identify the plan with the minimum total cost.

5. Pseudocode

```text
START
  SET usage_data = 50 GB, usage_calls = 3000 Mins (for 28
days)
  SET plans = [PlanA_details, PlanB_details, PlanC_details]
  FOR EACH plan IN plans DO
    extra_data = MAX(0, usage_data - plan.data_limit)
    extra_calls = MAX(0, usage_calls - plan.call_limit)
    total_cost = plan.rental + (extra_data * plan.data_rate) +
(extra_calls * plan.call_rate)
    STORE total_cost
  ENDFOR

  FIND plan with MIN(total_cost)
  PRINT recommended plan
END
```

6. Python Source Code

```python
def optimize_telecom_plan(total_data_gb, total_call_mins):
    days = 28
    plans = [
        {"name": "Plan A", "rental": 299, "data_limit": 1.5 * days,
"call_limit": 99999, "data_rate": 10, "call_rate": 0},
        {"name": "Plan B", "rental": 399, "data_limit": 2.0 * days,
"call_limit": 99999, "data_rate": 8, "call_rate": 0},
        {"name": "Plan C", "rental": 199, "data_limit": 1.0 * days,
"call_limit": 100 * days, "data_rate": 15, "call_rate": 0.50}
    ]
    best_plan = None
    min_cost = float('inf')
    results = []
    for p in plans:
        extra_data = max(0.0, total_data_gb - p["data_limit"])
        extra_calls = max(0.0, total_call_mins - p["call_limit"]
        cost = p["rental"] + (extra_data * p["data_rate"]) +
(extra_calls * p["call_rate"])
        results.append((p["name"], cost))

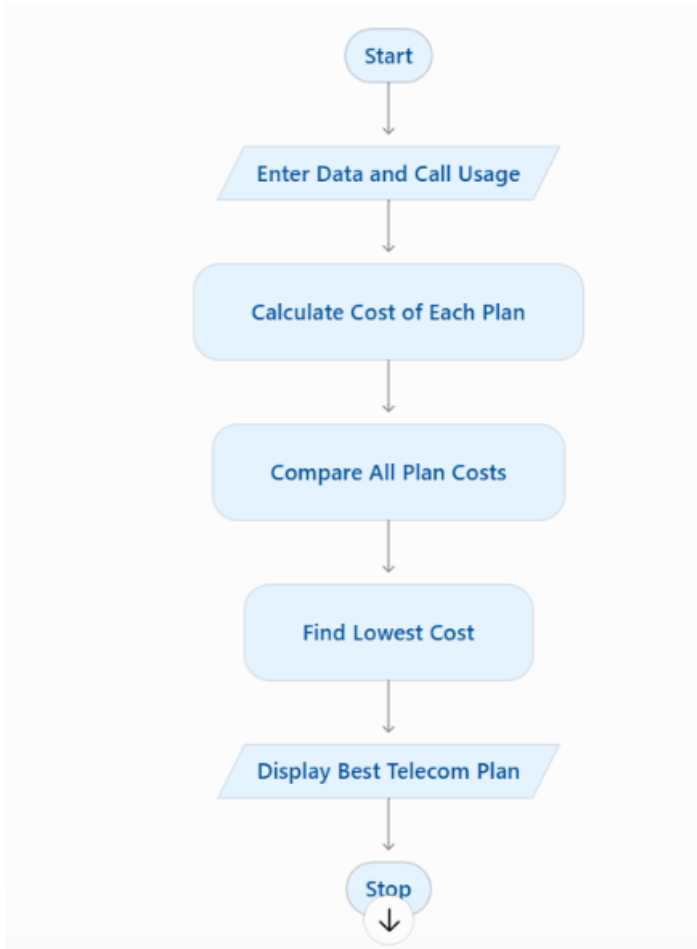
        if cost < min_cost:
            min_cost = cost
            best_plan = p["name"]
```

```
 return results, best_plan, min_cost

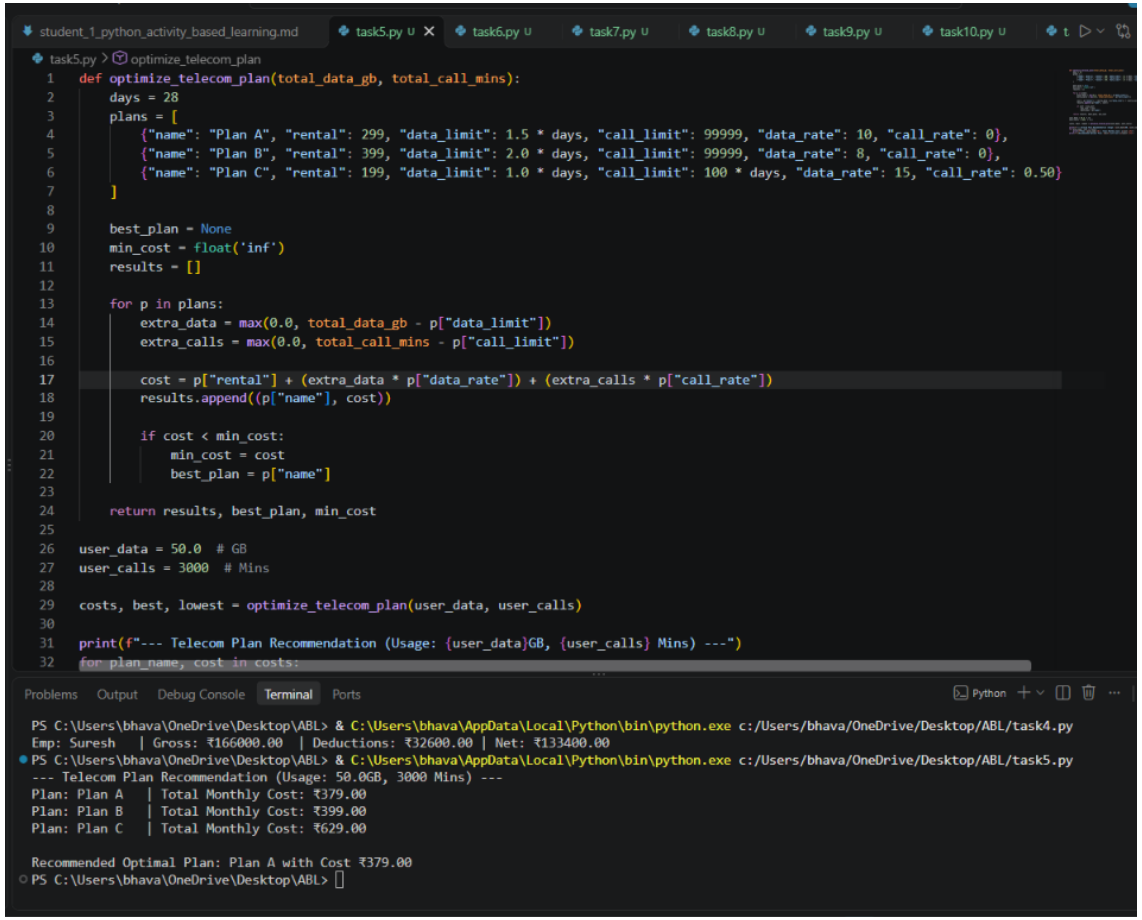

user_data = 50.0  # GB
user_calls = 3000  # Mins


costs, best, lowest = optimize_telecom_plan(user_data,
user_calls)


print(f"--- Telecom Plan Recommendation (Usage:
{user_data}GB, {user_calls} Mins) ---")
for plan_name, cost in costs:
    print(f"Plan: {plan_name:<8} | Total Monthly Cost:
₹{cost:.2f}")
print(f"\nRecommended Optimal Plan: {best} with Cost
₹{lowest:.2f}")
```


Start
Enter Data and Call Usage
Calculate Cost of Each Plan
Compare All Plan Costs
Find Lowest Cost
Display Best Telecom Plan
Stop

Screen shot code and output



8. Conclusion

The telecom cost optimizer evaluates potential overage fees across multi-tier mobile subscriptions to find the cheapest options for a user's usage patterns.

Case Study 6: Railway Fare and Reservation Calculator

1. Problem Statement

- Calculate total railway fares for multiple passengers incorporating distance slabs, travel classes, age-based concessions, reservation fees, and cancellation rules.

2. Objectives

- - Apply distance and class multiplier rules.
- - Integrate concessions (Senior Citizen, Child discount).
- - Calculate total ticket price with booking fees.

3. Assumptions

- - Base rate: ₹0.50 per km.
- - Class Multipliers: Sleeper = 1.0, 3AC = 2.5, 2AC = 3.5, 1AC = 5.0.
- - Concessions: Child (< 12 yrs) = 50% discount, Senior Citizen (≥ 60 yrs) = 40% discount.
- - Fixed Reservation Fee: ₹40 per passenger.

4. Algorithm

1. Calculate Base Fare = Distance * Base Rate * Class Multiplier.
2. For each passenger, calculate concession percentage based on age.
3. Compute Passenger Fare = Base Fare * (1 - Concession) + Reservation Fee.
4. Total Booking Fare = Sum of all individual passenger fares.

5. Pseudocode

```text
START
  READ distance, travel_class, list_of_passengers_ages

  multiplier = GET_MULTIPLIER(travel_class)
  base_fare = distance * 0.50 * multiplier
  total_bill = 0

  FOR EACH age IN list_of_passengers_ages DO
    IF age < 12 THEN
      fare = base_fare * 0.50
    ELSE IF age >= 60 THEN
      fare = base_fare * 0.60
    ELSE
      fare = base_fare

   ENDIF
        fare = fare + 40 (Reservation Fee)
        total_bill = total_bill + fare
     ENDFOR
     PRINT total_bill
   END
```

6. Python Source Code

```python
def calculate_railway_fare(distance_km, travel_class,
passengers):
    class_multipliers = {"Sleeper": 1.0, "3AC": 2.5, "2AC": 3.5,
"1AC": 5.0}
    reservation_fee = 40.0

    mult = class_multipliers.get(travel_class, 1.0)
    base_fare = distance_km * 0.50 * mult

    total_fare = 0.0
    details = []

    for name, age in passengers:
        if age < 12:
            discount = 0.50
```

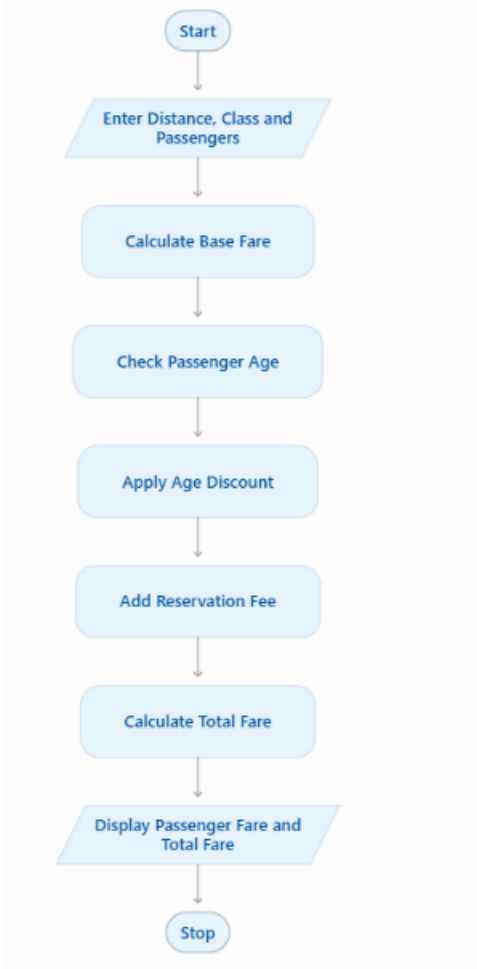
```
elif age >= 60:
   discount = 0.40
else:
   discount = 0.0
 p_fare = (base_fare * (1.0 - discount)) + reservation_fee
       total_fare += p_fare
       details.append((name, age, p_fare))

    return total_fare, details
passengers_list = [("Rajesh", 45), ("Sunita", 62), ("Rohan", 8)]
dist = 750  # km
t_class = "3AC"

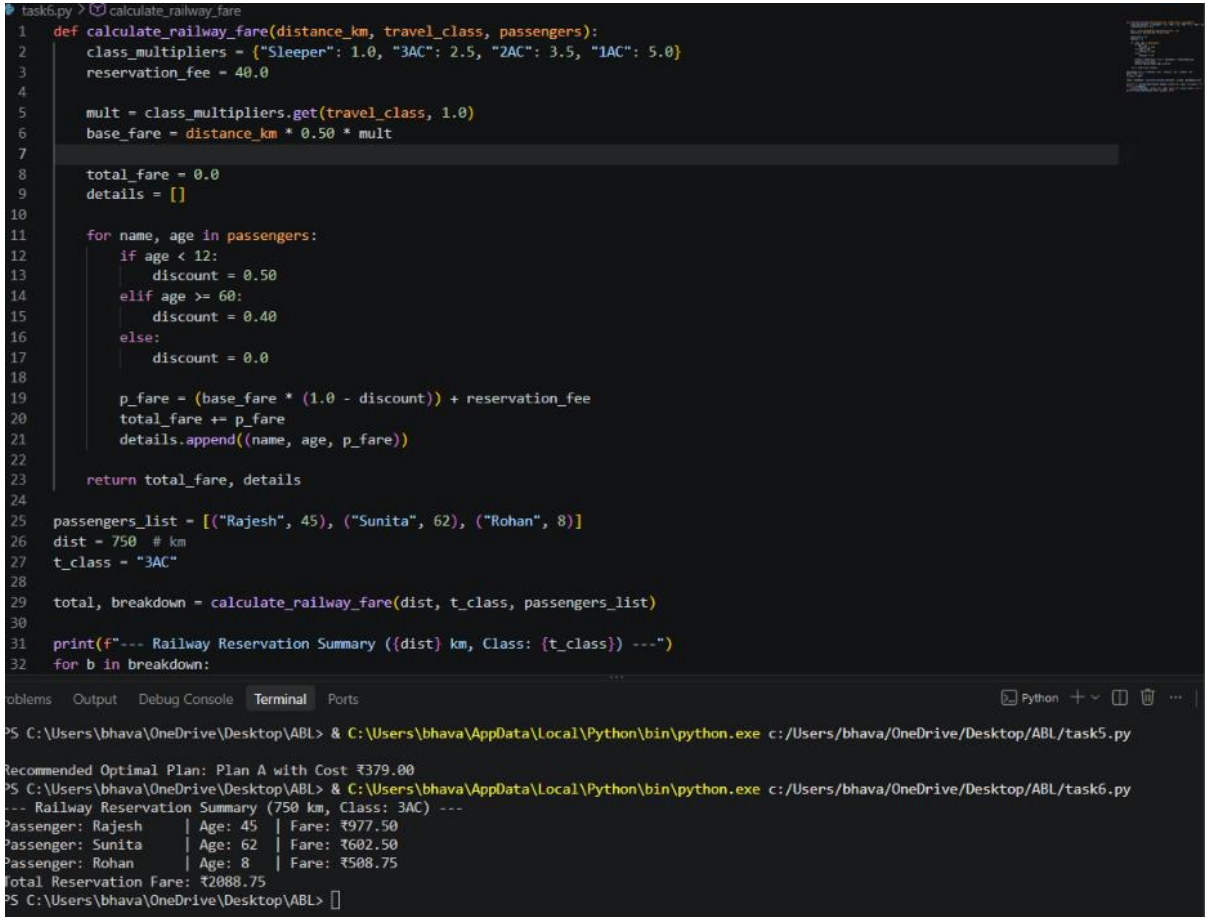
total, breakdown = calculate_railway_fare(dist, t_class,
passengers_list)



print(f"--- Railway Reservation Summary ({dist} km, Class:
{t_class}) ---")
for b in breakdown:
    print(f"Passenger: {b[0]:<10} | Age: {b[1]:<3} | Fare:
₹{b[2]:.2f}")
print(f"Total Reservation Fare: ₹{total:.2f}")
```


Start
Enter Distance, Class and Passengers
Calculate Base Fare
Check Passenger Age
Apply Age Discount
Add Reservation Fee
Calculate Total Fare
Display Passenger Fare and Total Fare
Stop

Screen shot code and output



```python
task6.py > calculate_railway_fare
def calculate_railway_fare(distance_km, travel_class, passengers):
    class_multipliers = {"Sleeper": 1.0, "3AC": 2.5, "2AC": 3.5, "1AC": 5.0}
    reservation_fee = 40.0

    mult = class_multipliers.get(travel_class, 1.0)
    base_fare = distance_km * 0.50 * mult

    total_fare = 0.0
    details = []

    for name, age in passengers:
        if age < 12:
            discount = 0.50
        elif age >= 60:
            discount = 0.40
        else:
            discount = 0.0

        p_fare = (base_fare * (1.0 - discount)) + reservation_fee
        total_fare += p_fare
        details.append((name, age, p_fare))

    return total_fare, details

passengers_list = [("Rajesh", 45), ("Sunita", 62), ("Rohan", 8)]
dist = 750  # km
t_class = "3AC"

total, breakdown = calculate_railway_fare(dist, t_class, passengers_list)

print(f"--- Railway Reservation Summary ({dist} km, Class: {t_class}) ---")
for b in breakdown:
```

```
oblems  Output  Debug Console  Terminal  Ports                                                    Python

S C:\Users\bhava\OneDrive\Desktop\ABL> & C:\Users\bhava\AppData\Local\Python\bin\python.exe c:/Users/bhava/OneDrive/Desktop/ABL/task5.py

ecommended Optimal Plan: Plan A with Cost ₹379.00
S C:\Users\bhava\OneDrive\Desktop\ABL> & C:\Users\bhava\AppData\Local\Python\bin\python.exe c:/Users/bhava/OneDrive/Desktop/ABL/task6.py
-- Railway Reservation Summary (750 km, Class: 3AC) ---
assenger: Rajesh     | Age: 45  | Fare: ₹977.50
assenger: Sunita     | Age: 62  | Fare: ₹602.50
assenger: Rohan      | Age: 8   | Fare: ₹508.75
otal Reservation Fare: ₹2088.75
S C:\Users\bhava\OneDrive\Desktop\ABL>
```

8. Conclusion

The script accurately computes distance-based rail fares along with category specific concessions and mandatory reservation charges.

Case Study 7: Retail Billing with Multiple Discount Rules

- **1. Problem Statement**
- Process itemized purchases with variable product quantities, category taxes, flat and percentage discounts, maximum caps, and final invoice formatting.
- **2. Objectives**
- - Compute subtotal per line item with applicable category GST/tax.
- - Apply conditional cart-level discounts with caps.
- - Output a clear, itemized retail tax invoice.

3. Assumptions

- - Category Taxes: Electronics = 18%, Grocery = 5%, Clothing = 12%.
- - Bulk Discount: 10% off on subtotal if cart subtotal $> ₹5000$ (Max discount ₹1000).

4. Algorithm

1. For each item (name, quantity, unit_price, category):
   a. Compute base item cost = quantity * unit_price.
   b. Compute tax = cost * tax_rate.
   c. Add item cost + tax to total cart value.
2. Check threshold discount condition. If cart basic subtotal > 5000, calculate 10% discount capped at ₹1000.
3. Compute final payable bill = Total Cart Value - Applied Discount.

5. Pseudocode

```text
START
  SET cart = [items]
  SET grand_total = 0, total_tax = 0

  FOR EACH item IN cart DO
    item_base = item.qty * item.price
    tax = item_base * TAX_RATE(item.category)
    item_total = item_base + tax
    grand_total = grand_total + item_total
  ENDFOR

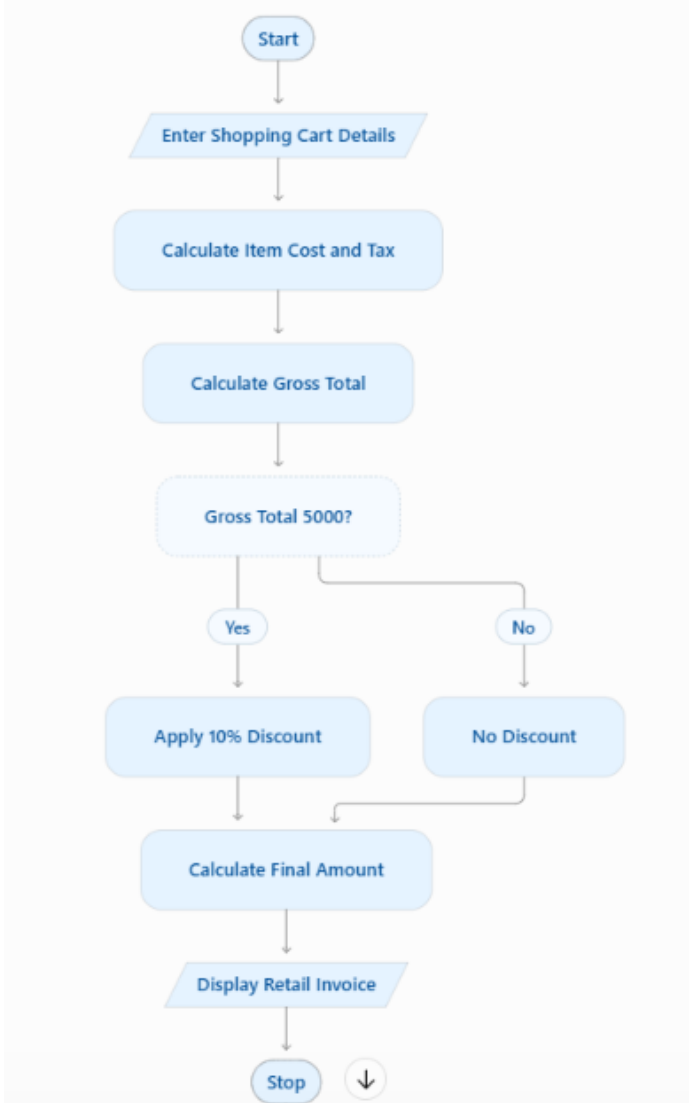
  IF grand_total > 5000 THEN
      discount = MIN(1000, 0.10 * grand_total)
    ELSE
      discount = 0
    ENDIF

    net_payable = grand_total - discount
    PRINT itemized summary and net_payable
  END
```

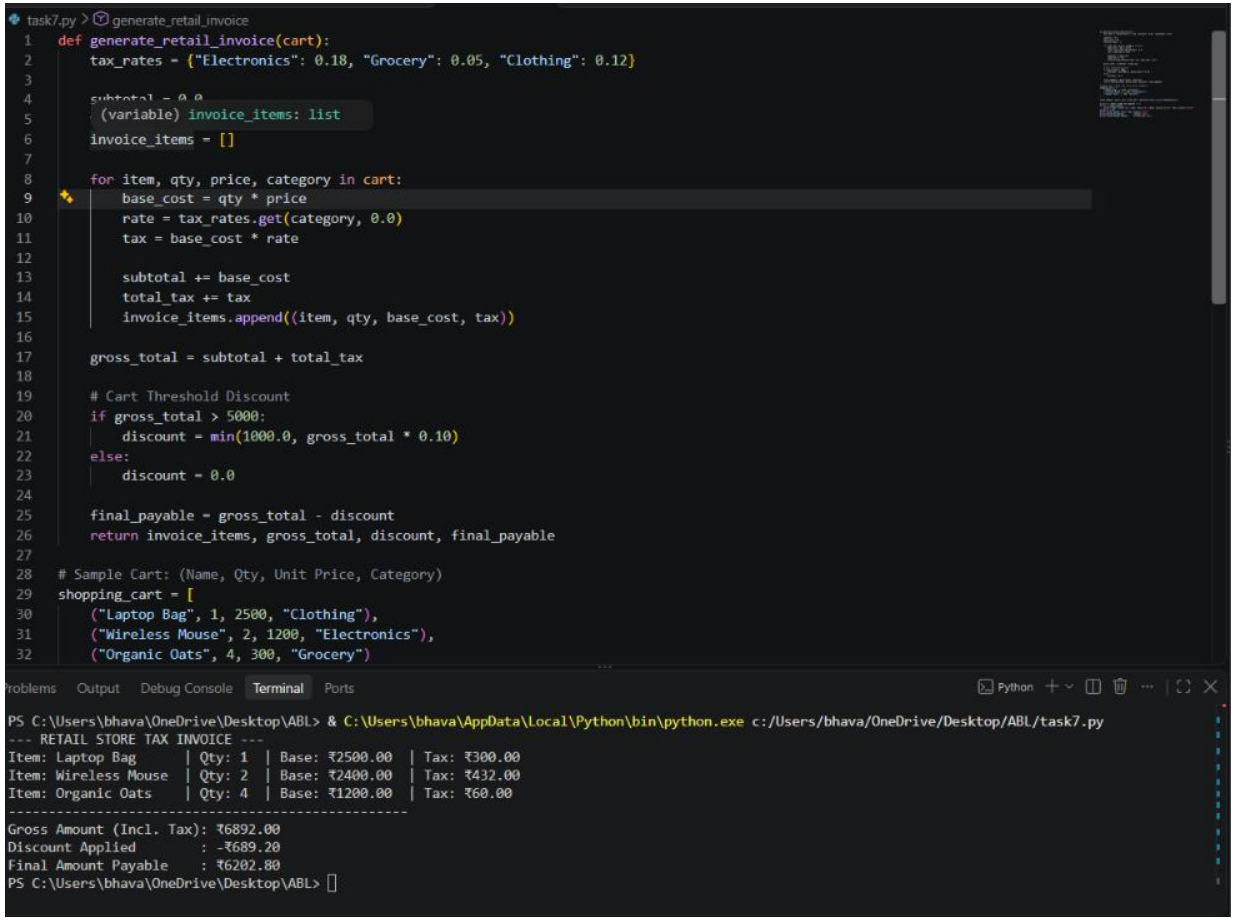
6. Python Source Code

```python
def generate_retail_invoice(cart):
    tax_rates = {"Electronics": 0.18, "Grocery": 0.05,
"Clothing": 0.12}
    subtotal = 0.0
    total_tax = 0.0
    invoice_items = []
    for item, qty, price, category in cart:
        base_cost = qty * price
        rate = tax_rates.get(category, 0.0)
        tax = base_cost * rate
        subtotal += base_cost
        total_tax += tax
        invoice_items.append((item, qty, base_cost, tax))
```

```
 gross_total = subtotal + total_tax
# Cart Threshold Discount
    if gross_total > 5000:
        discount = min(1000.0, gross_total * 0.10)
    else:
        discount = 0.0
    final_payable = gross_total - discount
    return invoice_items, gross_total, discount, final_payable
# Sample Cart: (Name, Qty, Unit Price, Category)
shopping_cart = [
    ("Laptop Bag", 1, 2500, "Clothing"),
    ("Wireless Mouse", 2, 1200, "Electronics"),
    ("Organic Oats", 4, 300, "Grocery")
]
items_summary, gross, disc, final_amt =
generate_retail_invoice(shopping_cart)
print("--- RETAIL STORE TAX INVOICE ---")
for it in items_summary:
    print(f"Item: {it[0]:<15} | Qty: {it[1]:<2} | Base: ₹{it[2]:<8.2f} |
Tax: ₹{it[3]:<6.2f}")
print("-" * 50)
print(f"Gross Amount (Incl. Tax): ₹{gross:.2f}")
print(f"Discount Applied       : -₹{disc:.2f}")
print(f"Final Amount Payable    : ₹{final_amt:.2f}")
```


Start
Enter Shopping Cart Details
Calculate Item Cost and Tax
Calculate Gross Total
Gross Total 5000?
Yes
No
Apply 10% Discount
No Discount
Calculate Final Amount
Display Retail Invoice
Stop

Screen shot code and output



```python
task7.py > generate_retail_invoice
def generate_retail_invoice(cart):
    tax_rates = {"Electronics": 0.18, "Grocery": 0.05, "Clothing": 0.12}

    subtotal = 0.0
    (variable) invoice_items: list
    invoice_items = []

    for item, qty, price, category in cart:
        base_cost = qty * price
        rate = tax_rates.get(category, 0.0)
        tax = base_cost * rate

        subtotal += base_cost
        total_tax += tax
        invoice_items.append((item, qty, base_cost, tax))

    gross_total = subtotal + total_tax

    # Cart Threshold Discount
    if gross_total > 5000:
        discount = min(1000.0, gross_total * 0.10)
    else:
        discount = 0.0

    final_payable = gross_total - discount
    return invoice_items, gross_total, discount, final_payable

# Sample Cart: (Name, Qty, Unit Price, Category)
shopping_cart = [
    ("Laptop Bag", 1, 2500, "Clothing"),
    ("Wireless Mouse", 2, 1200, "Electronics"),
    ("Organic Oats", 4, 300, "Grocery")
```

```
Problems  Output  Debug Console  Terminal  Ports                                Python

PS C:\Users\bhava\OneDrive\Desktop\ABL> & C:\Users\bhava\AppData\Local\Python\bin\python.exe c:/Users/bhava/OneDrive/Desktop/ABL/task7.py
--- RETAIL STORE TAX INVOICE ---
Item: Laptop Bag      | Qty: 1  | Base: ₹2500.00  | Tax: ₹300.00
Item: Wireless Mouse  | Qty: 2  | Base: ₹2400.00  | Tax: ₹432.00
Item: Organic Oats    | Qty: 4  | Base: ₹1200.00  | Tax: ₹60.00
--------------------------------------------------
Gross Amount (Incl. Tax): ₹6892.00
Discount Applied        : -₹689.20
Final Amount Payable    : ₹6202.80
PS C:\Users\bhava\OneDrive\Desktop\ABL>
```

8. Conclusion

The program calculates tax rates per category and applies discounts while capping maximum concessions.

Case Study 8: Manufacturing Cost and Profitability Analysis

- **1. Problem Statement**
- Calculate total production costs including raw material costs, labor fees, material wastage, machine operation fees, and sales returns. Determine whether production meets target profit margins.
-

2. Objectives

- - Aggregate direct and indirect operational costs.
- - Compute profit margins and compare against corporate target benchmarks.
-

3. Assumptions

- - Wastage overhead = 5% of Raw Material Cost.
- - Total Production Cost = Raw Material + Wastage + Labor + Machine Costs.
- - Profit Target = 20% margin over total cost.

4. Algorithm

1. Calculate raw material wastage = 0.05 * raw_material_cost.
2. Sum total cost = raw_material_cost + wastage + labor_cost + machine_cost.
3. Compute gross revenue = units_produced * selling_price_per_unit.
4. Profit = gross revenue - total cost.
5. Profit Margin % = (Profit / total cost) * 100.
6. Compare Profit Margin % against the 20% target.

5. Pseudocode

```text
START
  READ raw_mat_cost, labor_cost, machine_cost, units,
selling_price

  wastage = 0.05 * raw_mat_cost
  total_cost = raw_mat_cost + wastage + labor_cost +
machine_cost
  revenue = units * selling_price
  profit = revenue - total_cost
  margin_percent = (profit / total_cost) * 100

  IF margin_percent >= 20.0 THEN
    PRINT "Production Target Met"
  ELSE
    PRINT "Below Target Profit"
  ENDIF
END
```

6. Python Source Code

```python
def analyze_manufacturing_profitability(batch_id,
raw_material, labor, machine, units, unit_price):
    wastage_cost = 0.05 * raw_material
    total_production_cost = raw_material + wastage_cost +
labor + machine

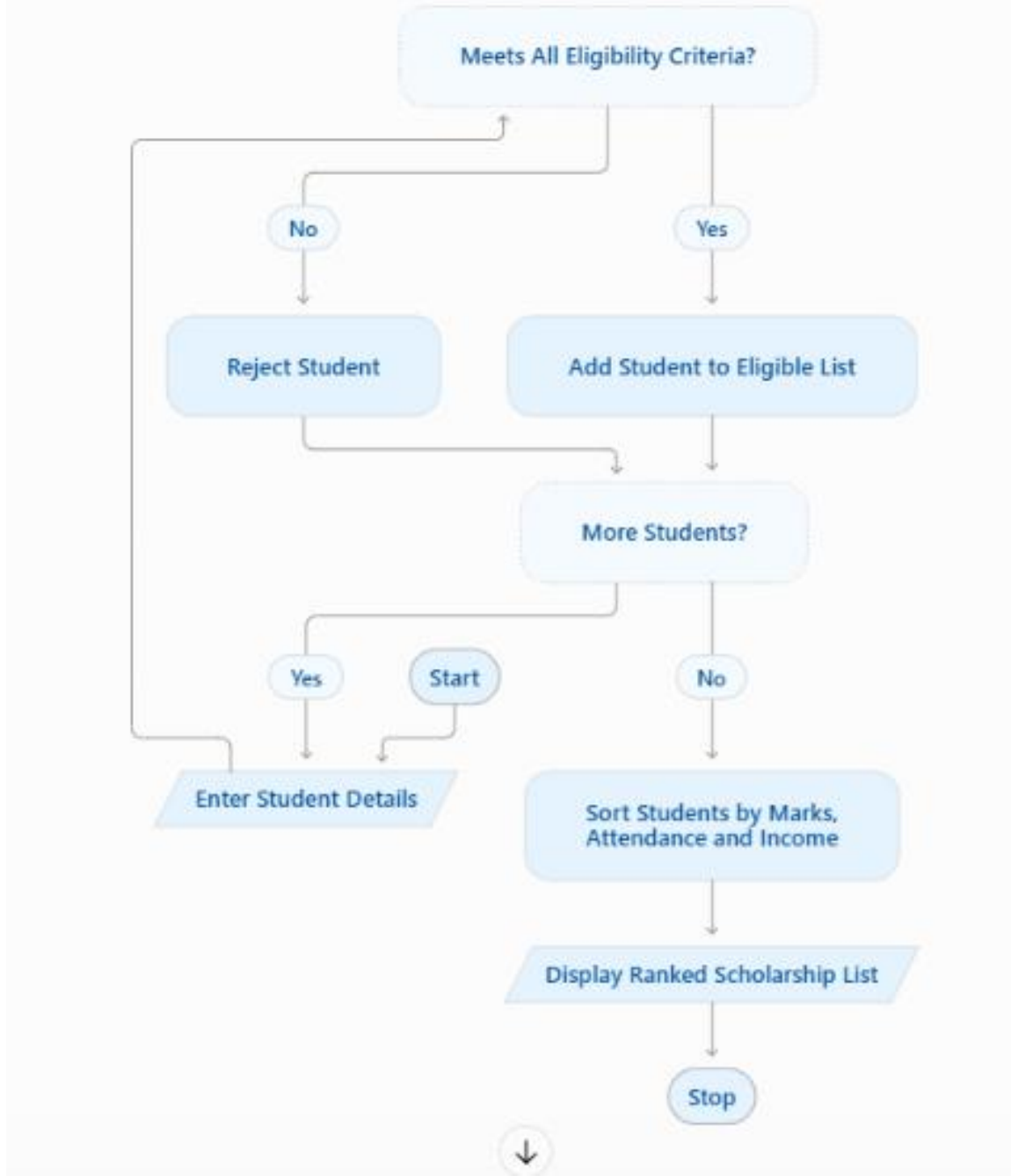
    gross_revenue = units * unit_price
    net_profit = gross_revenue - total_production_cost
    profit_margin = (net_profit / total_production_cost) * 100 if
total_production_cost > 0 else 0

    target_met = profit_margin >= 20.0
    return total_production_cost, gross_revenue, net_profit,
profit_margin, target_met
```

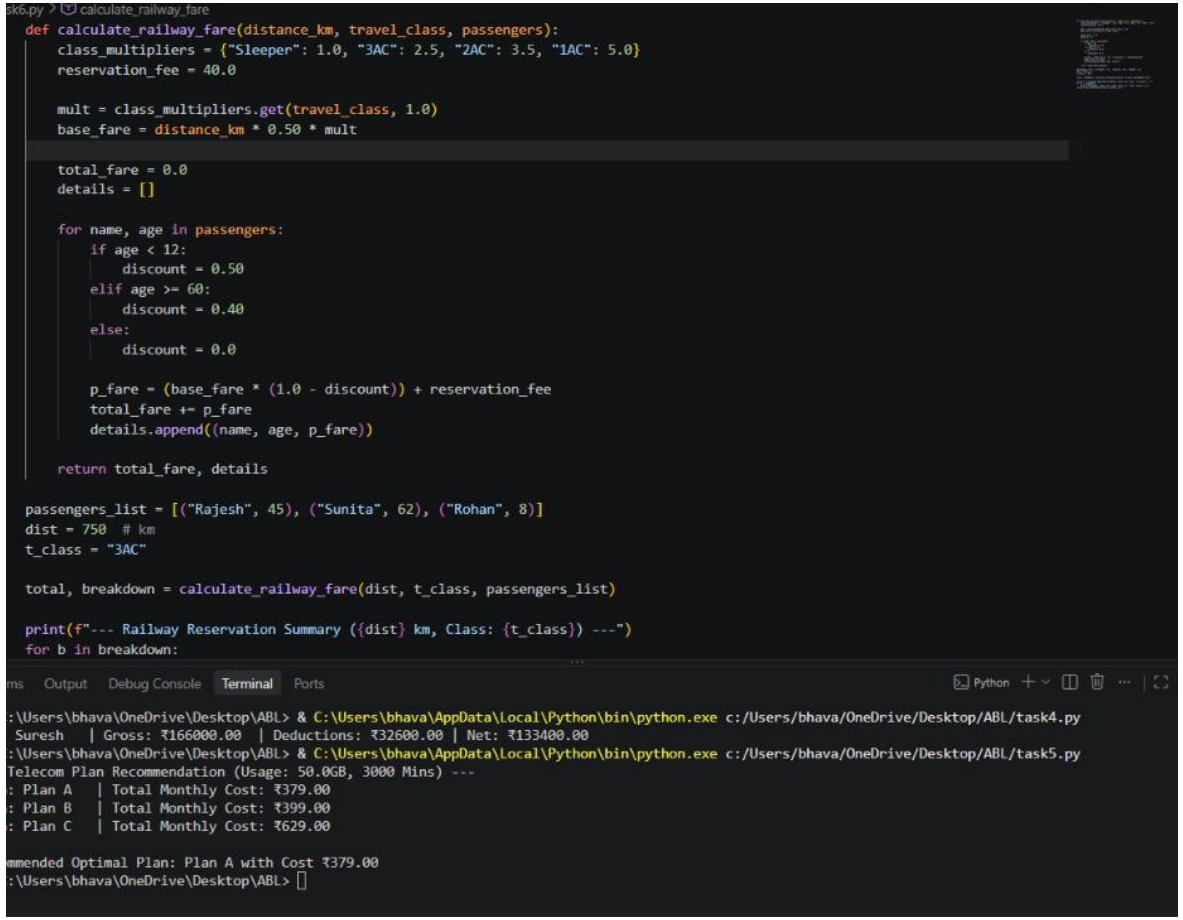
```
# Test Data
batch = "BATCH-2026-09"
raw_mat = 50000.0
labor = 20000.0
machine = 15000.0
units_produced = 1000
selling_price = 120.0


total_cost, revenue, profit, margin, status =
analyze_manufacturing_profitability(
    batch, raw_mat, labor, machine, units_produced,
selling_price
)


print(f"--- Manufacturing Profitability Analysis ({batch}) ---")
print(f"Total Production Cost : ₹{total_cost:.2f}")
print(f"Gross Revenue Realized : ₹{revenue:.2f}")
print(f"Net Profit Earned     : ₹{profit:.2f}")
print(f"Profit Margin         : {margin:.2f}%")
print(f"20% Profit Target Met : {'YES' if status else 'NO'}")
```


Meets All Eligibility Criteria?
No
Yes
Reject Student
Add Student to Eligible List
More Students?
Yes
Start
No
Enter Student Details
Sort Students by Marks, Attendance and Income
Display Ranked Scholarship List
Stop

Screen shot code and output



```
sk6.py > calculate_railway_fare
def calculate_railway_fare(distance_km, travel_class, passengers):
    class_multipliers = {"Sleeper": 1.0, "3AC": 2.5, "2AC": 3.5, "1AC": 5.0}
    reservation_fee = 40.0

    mult = class_multipliers.get(travel_class, 1.0)
    base_fare = distance_km * 0.50 * mult

    total_fare = 0.0
    details = []

    for name, age in passengers:
        if age < 12:
            discount = 0.50
        elif age >= 60:
            discount = 0.40
        else:
            discount = 0.0

        p_fare = (base_fare * (1.0 - discount)) + reservation_fee
        total_fare += p_fare
        details.append((name, age, p_fare))

    return total_fare, details

passengers_list = [("Rajesh", 45), ("Sunita", 62), ("Rohan", 8)]
dist = 750  # km
t_class = "3AC"

total, breakdown = calculate_railway_fare(dist, t_class, passengers_list)

print(f"--- Railway Reservation Summary ({dist} km, Class: {t_class}) ---")
for b in breakdown:
```

```
ms  Output  Debug Console  Terminal  Ports                                                    Python

:\Users\bhava\OneDrive\Desktop\ABL> & C:\Users\bhava\AppData\Local\Python\bin\python.exe c:/Users/bhava/OneDrive/Desktop/ABL/task4.py
 Suresh   | Gross: ₹166000.00  | Deductions: ₹32600.00 | Net: ₹133400.00
:\Users\bhava\OneDrive\Desktop\ABL> & C:\Users\bhava\AppData\Local\Python\bin\python.exe c:/Users/bhava/OneDrive/Desktop/ABL/task5.py
Telecom Plan Recommendation (Usage: 50.0GB, 3000 Mins) ---
: Plan A   | Total Monthly Cost: ₹379.00
: Plan B   | Total Monthly Cost: ₹399.00
: Plan C   | Total Monthly Cost: ₹629.00

mmended Optimal Plan: Plan A with Cost ₹379.00
:\Users\bhava\OneDrive\Desktop\ABL>
```

8. Conclusion

The analytical engine identifies expenditure overheads and measures whether actual batch yield fulfills corporate return targets.

Case Study 9: Multi-Currency Travel Budget Analysis

- **1. Problem Statement**
- Convert international travel expenses in foreign currencies to base currency incorporating exchange rate transaction conversion fees. Compare overall spending against total budget limits.

2. Objectives

- - Perform multi-currency conversions using rate dictionaries/lists.
- - Apply dynamic conversion bank fee surcharges.
- - Categorize spending and identify the largest cost driver.

3. Assumptions

- - Base Currency: INR (₹).
- - Exchange Rates: 1 USD = ₹83.0, 1 EUR = ₹90.0, 1 GBP = ₹105.0.
- - Standard bank foreign currency conversion charge = 2.5%.

4. Algorithm

1. Loop over expenses list containing (Category, Amount, Currency).
2. Convert amount to INR using the current exchange rate.
3. Calculate fee = Converted INR * 0.025. Total cost = Converted INR + Fee.
4. Accumulate total trip spending in INR.
5. Identify the largest expense category.
6. Compare overall spending with the user budget threshold.

5. Pseudocode

```text
START
  SET budget_inr = 200000
  SET expenses = [("Hotel", 800, "USD"), ("Food", 400,
"EUR"), ("Transit", 150, "GBP")]
  SET total_inr = 0

  FOR EACH item IN expenses DO
    inr_val = item.amount *
GET_EXCHANGE_RATE(item.currency)
    fee = inr_val * 0.025
    cost = inr_val + fee
    total_inr = total_inr + cost
    UPDATE max_category IF cost > max_cost
 ENDFOR

   PRINT total_inr, max_category
   IF total_inr > budget_inr THEN PRINT "Over Budget
 Warning" ENDIF
 END
```

6. Python Source Code

```python
def analyze_travel_budget(budget_inr, expense_records):
    rates = {"USD": 83.0, "EUR": 90.0, "GBP": 105.0, "INR": 1.0}
    conversion_fee_rate = 0.025

    total_spent_inr = 0.0
    highest_cost = 0.0
    largest_category = ""

    print("--- Foreign Expense Itemized Log (in INR) ---")
    for category, amount, currency in expense_records:
        rate = rates.get(currency, 1.0)
        base_inr = amount * rate
        fee = base_inr * conversion_fee_rate
        total_cost = base_inr + fee
```

```
total_spent_inr += total_cost

        if total_cost > highest_cost:
            highest_cost = total_cost
            largest_category = category

        print(f"Cat: {category:<12} | {amount:>7.2f} {currency:<3}
-> ₹{total_cost:<10.2f} (Incl. Fee)")

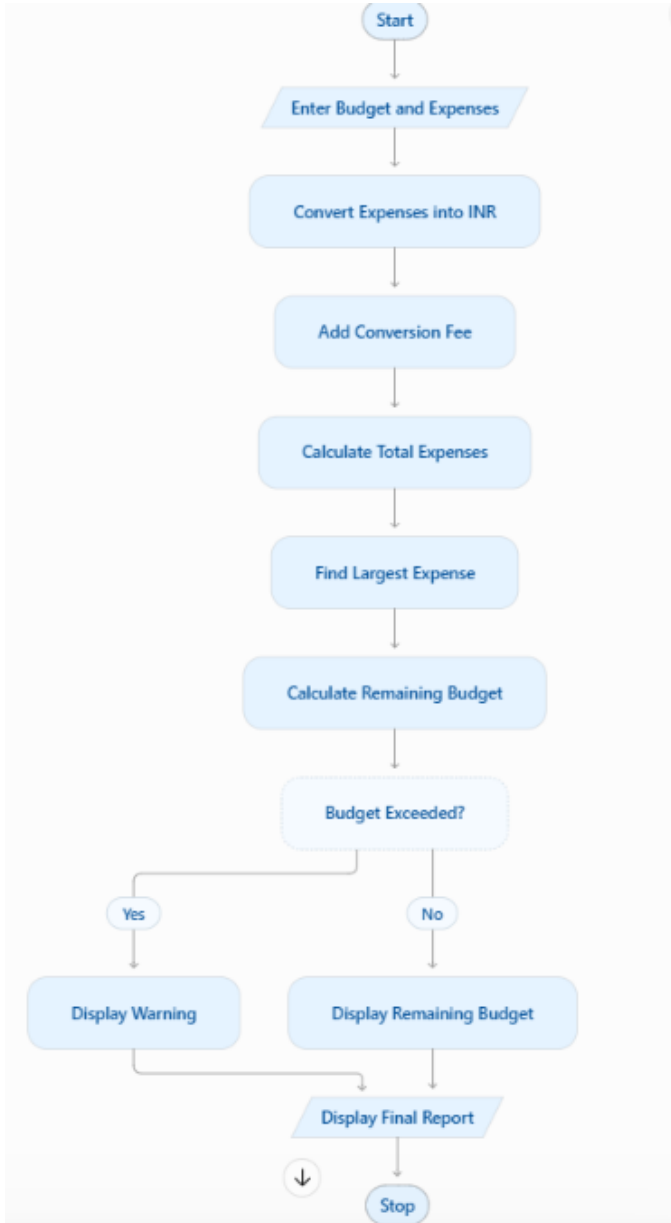
    balance = budget_inr - total_spent_inr
    return total_spent_inr, balance, largest_category



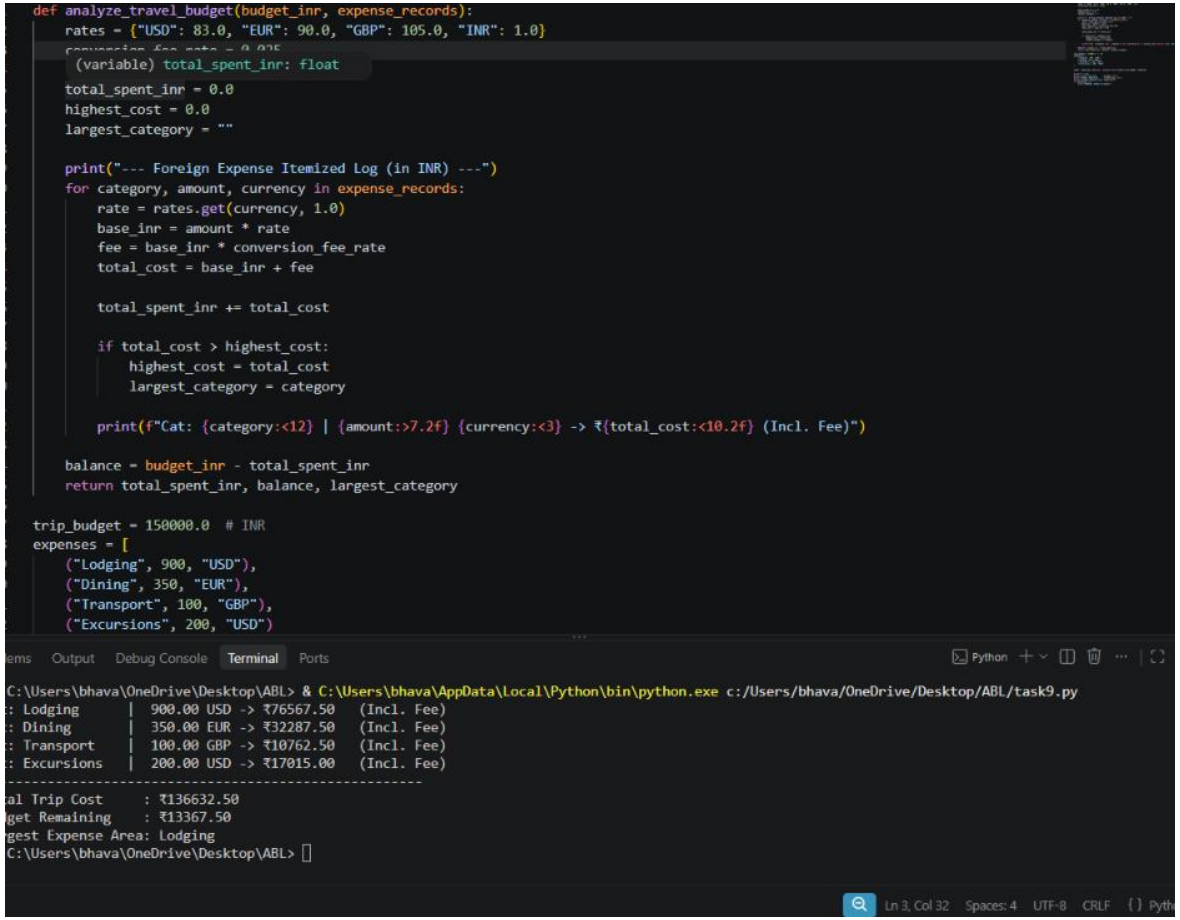
trip_budget = 150000.0  # INR
expenses = [
    ("Lodging", 900, "USD"),
    ("Dining", 350, "EUR"),
    ("Transport", 100, "GBP"),
    ("Excursions", 200, "USD")
]


spent, remaining, peak_cat =
analyze_travel_budget(trip_budget, expenses)
```

```
print("-" * 55)
print(f"Total Trip Cost     : ₹{spent:.2f}")
print(f"Budget Remaining    : ₹{remaining:.2f}")
print(f"Largest Expense Area: {peak_cat}")
if remaining < 0:
    print("WARNING: Budget Exceeded!")
```


Start
Enter Budget and Expenses
Convert Expenses into INR
Add Conversion Fee
Calculate Total Expenses
Find Largest Expense
Calculate Remaining Budget
Budget Exceeded?
Yes
No
Display Warning
Display Remaining Budget
Display Final Report
Stop

Screen shot code and output



```
def analyze_travel_budget(budget_inr, expense_records):
    rates = {"USD": 83.0, "EUR": 90.0, "GBP": 105.0, "INR": 1.0}
    conversion_fee_rate = 0.025
    (variable) total_spent_inr: float
    total_spent_inr = 0.0
    highest_cost = 0.0
    largest_category = ""

    print("--- Foreign Expense Itemized Log (in INR) ---")
    for category, amount, currency in expense_records:
        rate = rates.get(currency, 1.0)
        base_inr = amount * rate
        fee = base_inr * conversion_fee_rate
        total_cost = base_inr + fee

        total_spent_inr += total_cost

        if total_cost > highest_cost:
            highest_cost = total_cost
            largest_category = category

        print(f"Cat: {category:<12} | {amount:>7.2f} {currency:<3} -> ₹{total_cost:<10.2f} (Incl. Fee)")

    balance = budget_inr - total_spent_inr
    return total_spent_inr, balance, largest_category

trip_budget = 150000.0  # INR
expenses = [
    ("Lodging", 900, "USD"),
    ("Dining", 350, "EUR"),
    ("Transport", 100, "GBP"),
    ("Excursions", 200, "USD")
```

```
ems  Output  Debug Console  Terminal  Ports                                                    Python

C:\Users\bhava\OneDrive\Desktop\ABL> & C:\Users\bhava\AppData\Local\Python\bin\python.exe c:/Users/bhava/OneDrive/Desktop/ABL/task9.py
: Lodging      |  900.00 USD -> ₹76567.50   (Incl. Fee)
: Dining       |  350.00 EUR -> ₹32287.50   (Incl. Fee)
: Transport    |  100.00 GBP -> ₹10762.50   (Incl. Fee)
: Excursions   |  200.00 USD -> ₹17015.00   (Incl. Fee)
--------------------------------------------------
al Trip Cost     : ₹136632.50
get Remaining    : ₹13367.50
gest Expense Area: Lodging
C:\Users\bhava\OneDrive\Desktop\ABL>

Ln 3, Col 32  Spaces: 4  UTF-8  CRLF  { } Pyth
```

8. Conclusion

The travel budget system converts multi-currency payments, calculates fee overheads, tracks spending against limits, and highlights key cost drivers.

Case Study 10: Campus Event Financial Planning

- **1. Problem Statement**
- Calculate venue, equipment, catering, transport, and certificate expenses for a campus festival. Determine required student contribution fees, sponsorship gaps, and the breakeven attendance threshold.
- **2. Objectives**
- - Sum total variable and fixed event cost parameters.
- - Calculate required participant registration fees to reach economic break-even.
- **3. Assumptions**
- - Fixed Costs: Venue = ₹25,000, Sound & Lighting = ₹15,000.
- - Variable Costs per Attendee: Food/Catering = ₹200, Kit/Certificate = ₹50.
- - External Sponsorship Raised = ₹20,000.
- - Proposed Ticket Entry Fee = ₹300 per participant.

4. Algorithm

1. Total Fixed Cost = Venue Cost + Sound Cost.
2. Per Participant Contribution Margin = Ticket Fee - Variable Cost per attendee.
3. Net Fixed Cost to cover = Total Fixed Cost - Sponsorship.
4. Breakeven Attendance Count = Ceil(Net Fixed Cost / Per Participant Contribution Margin).
5. Output cost breakdowns and breakeven ticket sales requirement.

5. Pseudocode

```text
START
   SET fixed_cost = 25000 + 15000
   SET var_cost_per_head = 200 + 50
   SET ticket_price = 300
   SET sponsorship = 20000

   net_fixed = fixed_cost - sponsorship
   margin_per_head = ticket_price - var_cost_per_head

   breakeven_count = INT(net_fixed / margin_per_head)

   PRINT "Breakeven Participants Required:",
breakeven_count
END
```

6. Python Source Code

```python
import math



def calculate_event_breakeven(venue, equipment,
catering_per_head, cert_per_head, ticket_price,
sponsorship):
    fixed_expenses = venue + equipment
    variable_cost_per_head = catering_per_head +
cert_per_head

    contribution_margin = ticket_price -
variable_cost_per_head

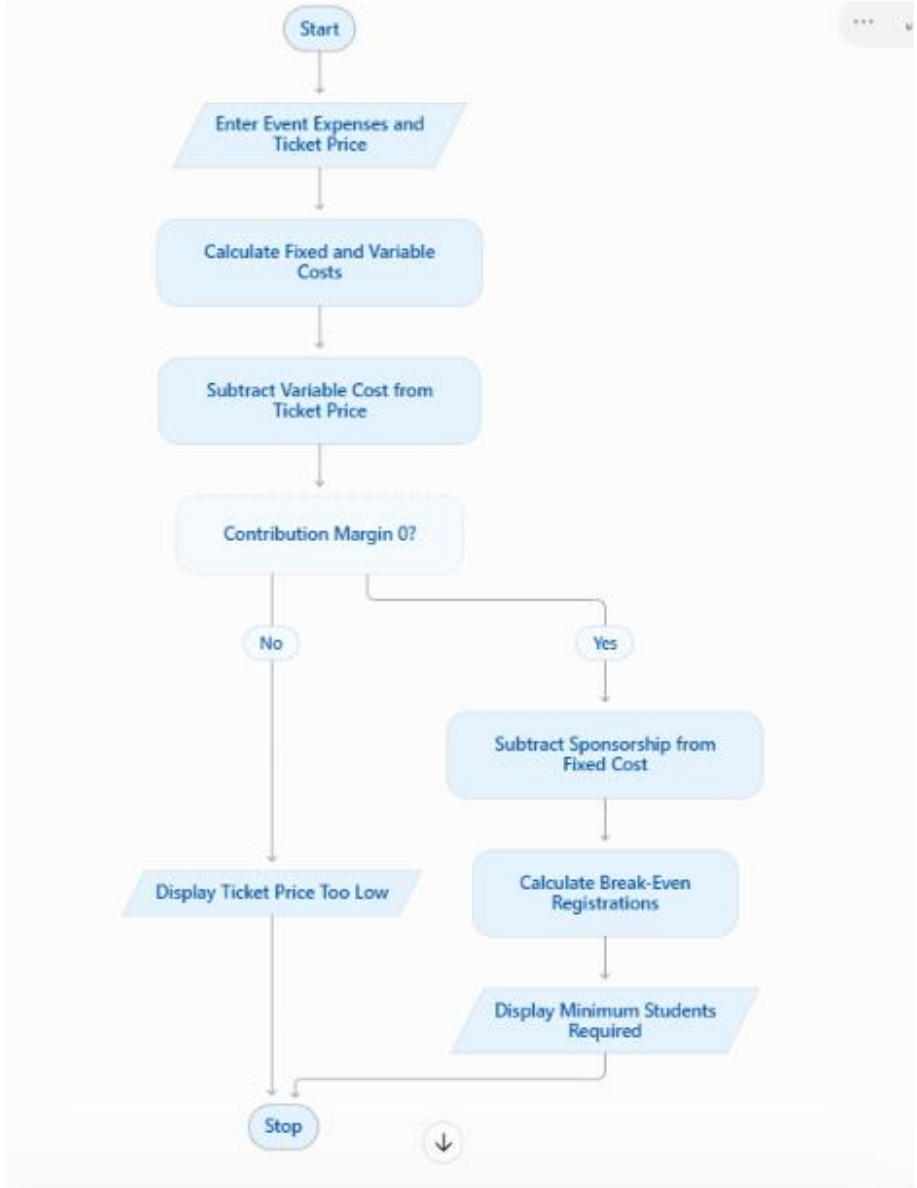
    if contribution_margin <= 0:
        return "Ticket price too low to cover variable costs per
head."
    net_fixed_to_cover = fixed_expenses - sponsorship

    if net_fixed_to_cover <= 0:
        breakeven_attendees = 0
```

```
else:
      breakeven_attendees = math.ceil(net_fixed_to_cover /
contribution_margin)

   return fixed_expenses, net_fixed_to_cover,
breakeven_attendees
  v_cost = 25000.0
  eq_cost = 15000.0
  cat_head = 200.0
  cert_head = 50.0
  price = 350.0
  sponsor = 18000.0

  total_f, net_f, count = calculate_event_breakeven(v_cost,
  eq_cost, cat_head, cert_head, price, sponsor)
  print("--- Campus Event Financial Planning ---")
  print(f"Total Fixed Event Overhead : ₹{total_f:.2f}")
  print(f"Sponsorship Offset Covered : ₹{sponsor:.2f}")
  print(f"Net Fixed Cost Remaining   : ₹{net_f:.2f}")
  print(f"Ticket Price per Head      : ₹{price:.2f}")
  print(f"Variable Cost per Head     : ₹{cat_head +
  cert_head:.2f}")
  print(f"Minimum Registrations Needed for Break-Even:
  {count} Students")
```


Start
Enter Event Expenses and Ticket Price
Calculate Fixed and Variable Costs
Subtract Variable Cost from Ticket Price
Contribution Margin 0?
No
Yes
Subtract Sponsorship from Fixed Cost
Display Ticket Price Too Low
Calculate Break-Even Registrations
Display Minimum Students Required
Stop

task10.py > ...

```python
import math

def calculate_event_breakeven(venue, equipment, catering_per_head, cert_per_head, ticket_price, sponsorship):
    fixed_expenses = venue + equipment
    variable_cost_per_head = catering_per_head + cert_per_head

    contribution_margin = ticket_price - variable_cost_per_head

    if contribution_margin <= 0:
        return "Ticket price too low to cover variable costs per head."

    net_fixed_to_cover = fixed_expenses - sponsorship

    if net_fixed_to_cover <= 0:
        breakeven_attendees = 0
    else:
        breakeven_attendees = math.ceil(net_fixed_to_cover / contribution_margin)

    return fixed_expenses, net_fixed_to_cover, breakeven_attendees

v_cost = 25000.0
eq_cost = 15000.0
cat_head = 200.0
cert_head = 50.0
price = 350.0
sponsor = 18000.0

total_f, net_f, count = calculate_event_breakeven(v_cost, eq_cost, cat_head, cert_head, price, sponsor)

print("--- Campus Event Financial Planning ---")
print(f"Total Fixed Event Overhead : ₹{total_f:.2f}")
print(f"Sponsorship Offset Covered : ₹{sponsor:.2f}")
```

Problems Output Debug Console Terminal Ports Python

```
PS C:\Users\bhava\OneDrive\Desktop\ABL> & C:\Users\bhava\AppData\Local\Python\bin\python.exe c:/Users/bhava/OneDrive/Desktop/ABL/task9.py
PS C:\Users\bhava\OneDrive\Desktop\ABL> & C:\Users\bhava\AppData\Local\Python\bin\python.exe c:/Users/bhava/OneDrive/Desktop/ABL/task10.py
--- Campus Event Financial Planning ---
Total Fixed Event Overhead : ₹40000.00
Sponsorship Offset Covered : ₹18000.00
Net Fixed Cost Remaining   : ₹22000.00
Ticket Price per Head      : ₹350.00
Variable Cost per Head     : ₹250.00
Minimum Registrations Needed for Break-Even: 220 Students
PS C:\Users\bhava\OneDrive\Desktop\ABL>
```

8. Conclusion

The financial modeling code identifies minimum student participant targets needed to run event operations without experiencing a deficit.